Best Practices für Confluent Kafka

NetApp Solutions

NetApp
December 20, 2022

This PDF was generated from https://docs.netapp.com/de-de/netapp-solutions/data-analytics/confluent-kafka-introduction.html on December 20, 2022. Always check docs.netapp.com for the latest.

Inhaltsverzeichnis

Best Practices für Confluent Kafka . . . 1
TR-4912: Best Practice Guidelines für Conflient Kafka Tiered Storage mit NetApp . . . 1
Details zur Lösungsarchitektur . . . 3
Technologieüberblick . . . 4
Confluent Verification . . . 11
Performance-Tests mit Skalierbarkeit . . . 14
Fließender s3-Anschluss . . . 16
Confluent Self balancing Clusters . . . 25
Best Practices-Richtlinien . . . 25
Dimensionierung . . . 27
Schlussfolgerung . . . 30

Best Practices für Confluent Kafka

TR-4912: Best Practice Guidelines für Conflient Kafka Tiered Storage mit NetApp

Karthikeyan Nagalingam, Joseph Kandatilparambil, NetApp Rankesh Kumar, Confluent

Apache Kafka ist eine Community-verteilte Event-Streaming-Plattform, die täglich Billionen von Ereignissen bewältigen kann. Kafka wurde ursprünglich als Messaging-Warteschlange konzipiert und basiert auf einer Abstraktion eines verteilten Commit-Protokolls. Seit der Gründung und Open-sourcing von LinkedIn im Jahr 2011 hat sich Kafka von einer Nachrichtenwarteschlange zu einer vollwertigen Event-Streaming-Plattform entwickelt. Confluent liefert die Distribution von Apache Kafka mit der Confluent Platform. Die Confluent Platform ergänzt Kafka um zusätzliche Community- und kommerzielle Funktionen, die das Streamingerlebnis sowohl von Betreibern als auch von Entwicklern in der Produktion in großem Maßstab verbessern sollen.

Dieses Dokument beschreibt die Best Practice-Richtlinien für die Verwendung von Confluent Tiered Storage auf dem Objekt-Storage-Angebot von NetApp, indem es folgende Inhalte zur Verfügung stellt:

- Fließende Verifizierung mit NetApp Objekt-Storage – NetApp StorageGRID
- Tiered Storage-Performance-Tests
- Best Practice-Richtlinien für Conflient auf NetApp Storage-Systeme

Warum Confluent Tiered Storage?

Confluent hat sich zur standardmäßigen Streaming-Plattform in Echtzeit für viele Applikationen entwickelt. Dies gilt insbesondere für Big Data, Analysen und Streaming Workloads. Tiered Storage ermöglicht Benutzern die Trennung von Computing und Storage auf der Conflient Plattform. Mit sie lassen sich Daten kostengünstiger speichern, nahezu unbegrenzte Datenmengen speichern und On-Demand-Workloads skalieren. Administrative Aufgaben wie die Ausbalancierung von Daten und Mandanten werden vereinfacht. S3-kompatible Storage-Systeme profitieren von all diesen Funktionen, um Daten bei allen Ereignissen an einem Ort zu dezentralisieren. Dadurch ist ein komplexes Daten-Engineering überflüssig. Weitere Informationen dazu, warum Sie Tiered Storage für Kafka verwenden sollten, finden Sie hier "Dieser Artikel von Confluent".

Warum NetApp StorageGRID für Tiered Storage?

StorageGRID ist eine der Objekt-Storage-Plattformen von NetApp, StorageGRID ist eine softwaredefinierte, objektbasierte Storage-Lösung, die dem Branchenstandard entsprechende Objekt-APIs, einschließlich der S3 API (Amazon Simple Storage Service) unterstützt. StorageGRID speichert und managt unstrukturierte Daten jeder Größenordnung und stellt sicheren und langlebigen Objekt-Storage zur Verfügung. Inhalte werden zur richtigen Zeit am richtigen Ort und in der richtigen Storage-Tier platziert. Dies optimiert Workflows und senkt die Kosten für global verteilte Rich Media.

Das größte Unterscheidungsmerkmal von StorageGRID ist die Policy Engine für Information Lifecycle Management (ILM), die ein richtlinienbasiertes Management des Daten-Lebenszyklus ermöglicht. Die Richtlinien-Engine kann Metadaten verwenden, um zu managen, wie die Daten über ihre gesamte Lebensdauer gespeichert werden. So kann sie anfänglich auf eine optimale Performance aufsetzt und bei zunehmenden Alter der Daten automatisch auf Kosten und Langlebigkeit optimiert werden.

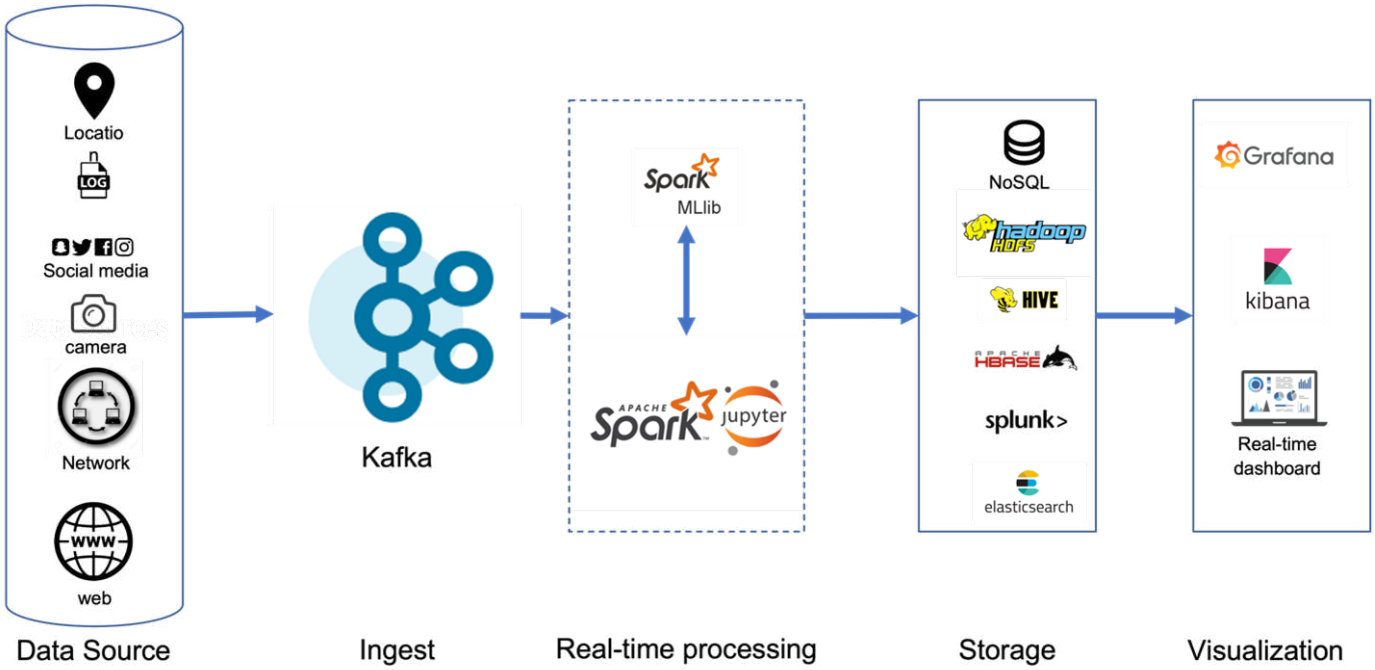
Aktivieren Von Confluent Tiered Storage

Die grundlegende Idee von Tiered Storage besteht darin, die Storage-Aufgaben von der Datenverarbeitung zu

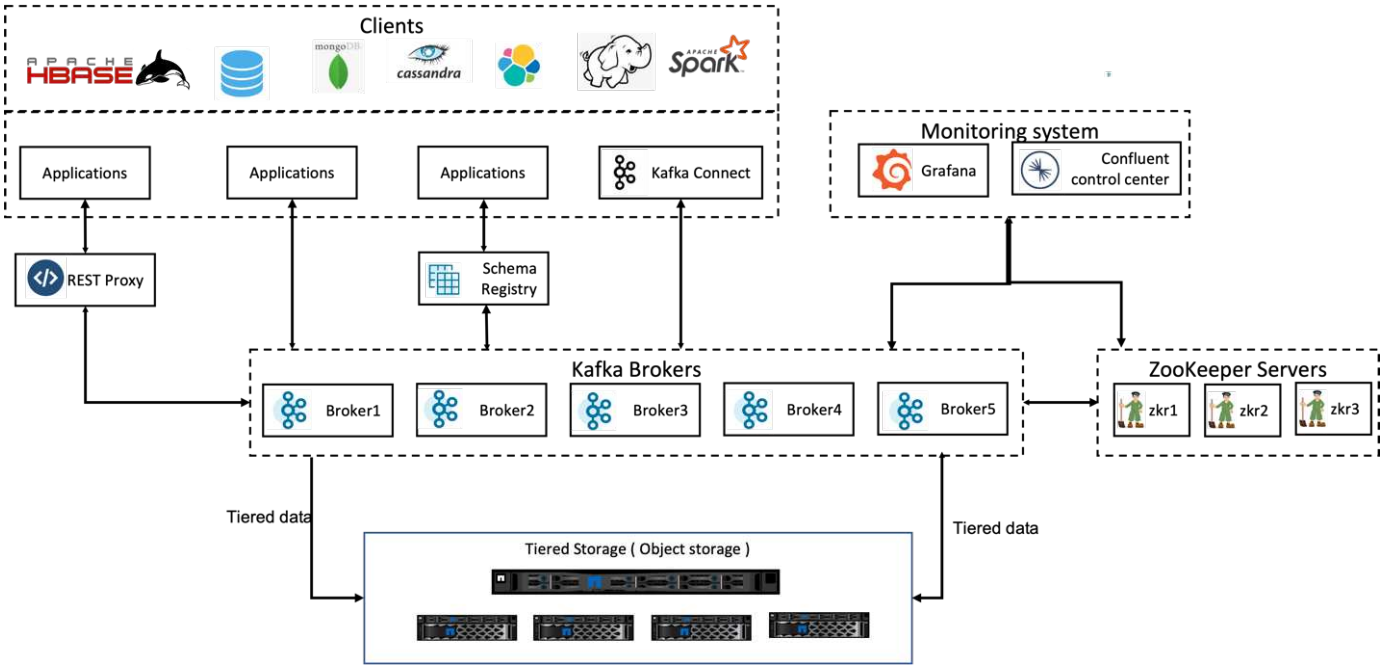
trennen. Dank dieser Trennung ist es viel einfacher, dass Storage Tiers und die Datenverarbeitungs-Tier unabhängig skaliert werden können.

Eine Tiered Storage-Lösung für Confluent muss zwei Faktoren berücksichtigen. Zunächst muss die IT sich um häufige Objekt-Storage-Konsistenz und Verfügbarkeiteigenschaften wie Inkonsistenzen bei LISTENVORGÄNGEN und gelegentlichen Nichtverfügbarkeit von Objekten handeln. Zweitens muss sie die Interaktion zwischen Tiered Storage und Kafkas Replizierungs- und Fehlertoleranz-Modell korrekt handhaben, einschließlich der Möglichkeit, dass Zombie-Führungskräfte weiterhin Tier-Offset-Bereiche nutzen. NetApp Objekt-Storage bietet sowohl die konsistente Objektverfügbarkeit als auch das HA-Modell, sodass der müde Storage für Tier-Offset-Bereiche verfügbar ist. NetApp Objekt-Storage bietet eine konsistente Objektverfügbarkeit sowie ein HA-Modell, um den müden Storage für Tier-Offset-Bereiche verfügbar zu machen.

Mit Tiered Storage können Sie hochperformante Plattformen für Lese- und Schreibvorgänge mit niedriger Latenz in Reichweite des Datenstreamings nutzen. Außerdem können Sie kostengünstigere und skalierbare Objektspeicher wie NetApp StorageGRID für historische Lesevorgänge mit hohem Durchsatz nutzen. Außerdem verfügen wir über technische Lösungen für Spark mit netapp Storage Controller und erhalten weitere Informationen. Die folgende Abbildung zeigt, wie Kafka in eine Echtzeit-Analyse-Pipeline passt.



Die folgende Abbildung zeigt, wie NetApp StorageGRID zu einem Objekt-Storage Tier von Conflient Kafka passt.



"Weiter: Details zur Lösungsarchitektur."

Details zur Lösungsarchitektur

"Zurück: Einführung."

Dieser Abschnitt behandelt die Hardware und Software, die für die Confluent-Überprüfung verwendet werden. Diese Informationen gelten für die Implementierung der Conflient Plattform mit NetApp Storage. Die folgende Tabelle deckt die getestete Lösungsarchitektur und Basiskomponenten ab.

Lösungskomponenten	Details
Conflient Kafka Version 6.2	• Drei Zookeeper • Fünf Broker Server • Fünf-Tools-Server • One Grafana • Ein Kontrollzentrum
Linux (ubuntu 18.04)	Alle Server
NetApp StorageGRID für Tiered Storage	• StorageGRID Software • 1 x SG1000 (Load Balancer) • 4 x SGF6024 • 4 x 24 x 800 SSDs • S3-Protokoll • 4 x 100 GbE (Netzwerkkonnektivität zwischen Broker und StorageGRID Instanzen)

Lösungskomponenten	Details
15 Fujitsu PRIMERGY RX2540 Server	Jeweils ausgestattet mit: * 2 CPUs, insgesamt 16 physischen Kernen * Intel Xeon * 256 GB physischem Speicher * 100 GbE Dual Port

"Weiter: Technologische Übersicht."

Technologieüberblick

"Zurück: Details zur Lösungsarchitektur."

In diesem Abschnitt wird die in dieser Lösung verwendete Technologie beschrieben.

NetApp StorageGRID

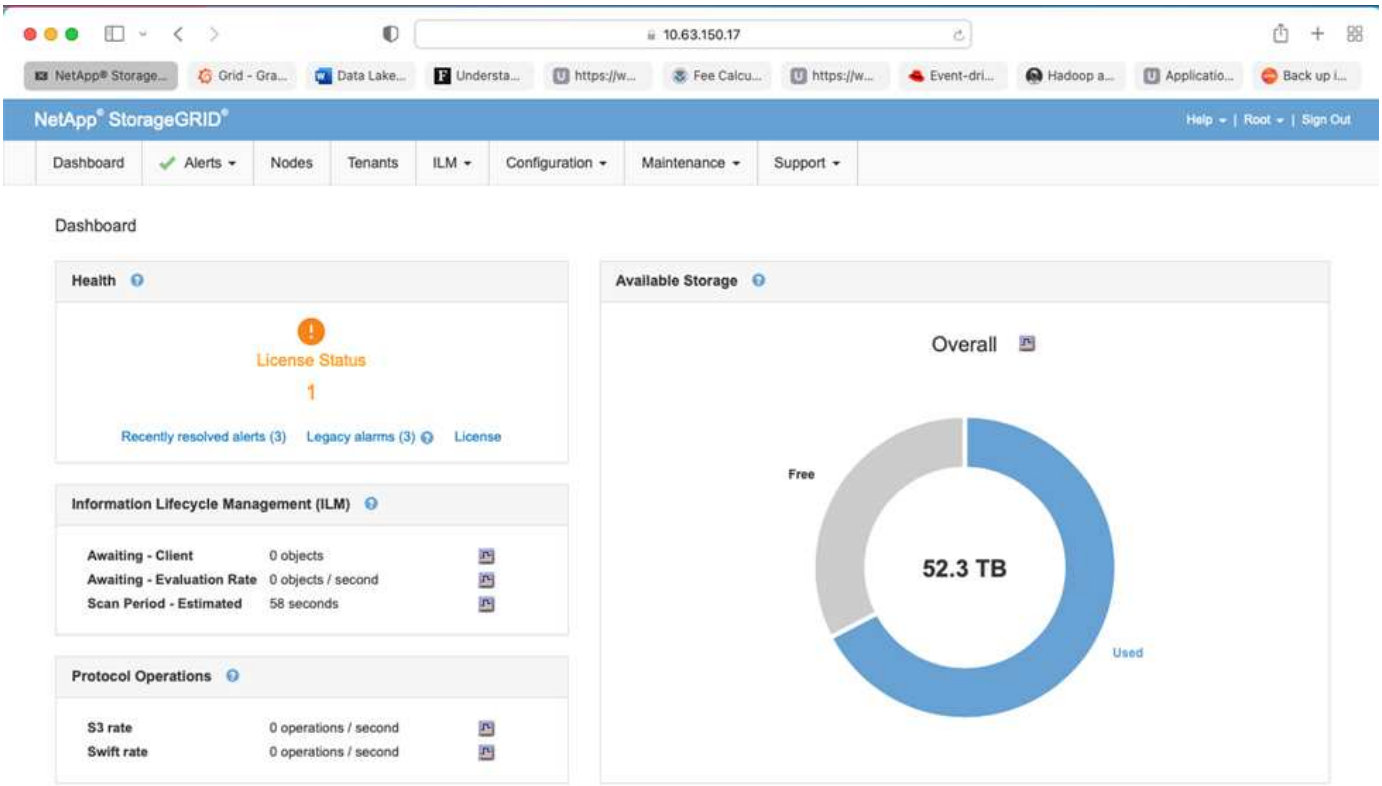
NetApp StorageGRID ist eine hochperformante, kostengünstige Objekt-Storage-Plattform. Mithilfe von Tiered Storage werden die meisten Daten von Conflient Kafka, die im lokalen Storage oder im SAN-Storage des Brokers gespeichert sind, in den Remote-Objektspeicher ausgelagert. Diese Konfiguration führt zu deutlichen betrieblichen Verbesserungen, da der Zeit- und Kostenaufwand für den Ausgleich, die Erweiterung, die Verkleinerung von Clustern oder den Austausch eines ausgefallenen Brokers verringert wird. Objekt-Storage spielt eine wichtige Rolle beim Management von Daten, die sich auf der Objektspeicher-Tier befinden. Aus diesem Grund ist die Wahl des richtigen Objekt-Storage wichtig.

StorageGRID bietet intelligentes, richtlinienbasiertes globales Datenmanagement unter Verwendung einer verteilten, Node-basierten Grid-Architektur. Dank des universellen globalen Objekt-Namespace, der komplexe Datenmanagement-Funktionen vereint, vereinfacht es das Management von unstrukturierten Daten im Petabyte-Bereich und Milliarden von Objekten. Der Zugriff auf einzelne Objekte ist über alle Standorte hinweg möglich und vereinfacht Hochverfügbarkeitsarchitekturen bei kontinuierlichem Objektzugriff, unabhängig von Standort- oder Infrastrukturausfällen.

Dank Mandantenfähigkeit können mehrere unstrukturierte Cloud- und Enterprise-Datenapplikationen sicher im selben Grid gewartet werden. Dies erhöht den ROI und die Anwendungsfälle für NetApp StorageGRID. Sie haben die Möglichkeit, diverse Service-Level mit metadatengestützten Objekt-Lebenszyklus-Richtlinien zu erstellen. So optimieren Sie Langlebigkeit, Schutz und Performance an mehreren Standorten und Regionen. Benutzer können Datenmanagement-Richtlinien anpassen sowie Traffic-Grenzen überwachen und anwenden, um sich unterbrechungsfrei an die Datenlandschaft anzupassen, wenn sich ihre Anforderungen ständig ändern.

Einfaches Management mit Grid Manager

Der StorageGRID Grid Manager ist eine browserbasierte grafische Oberfläche, mit der Sie Ihr StorageGRID System an weltweit verteilten Standorten über eine zentrale Konsole konfigurieren, managen und überwachen können.



Sie können die folgenden Aufgaben über die StorageGRID Grid Manager-Oberfläche ausführen:

- Verwalten global verteilter Repositorys mit mehreren Petabyte für Objekte wie Bilder, Videos und Datensätze.
- Grid-Nodes und -Services überwachen, um die Objektverfügbarkeit zu gewährleisten
- Mithilfe von ILM-Regeln (Information Lifecycle Management) managen Sie Objektdaten über einen längeren Zeitraum. Diese Regeln regeln, was mit den Objektdaten nach der Aufnahme geschieht, wie sie vor Verlust geschützt sind, wo Objektdaten gespeichert werden und wie lange.
- Überwachung von Transaktionen, Performance und Vorgängen im System

Richtlinien für das Information Lifecycle Management

StorageGRID verfügt über flexible Datenmanagement-Richtlinien. Dazu zählen die Aufbewahrung von Replikatkopien Ihrer Objekte und die Verwendung von EC-Schemata (Erasure Coding) wie 2+1 und 4+2 (u. a.) zum Speichern Ihrer Objekte, je nach spezifischen Performance- und Datensicherungsanforderungen. Wenn sich Workloads und Anforderungen im Laufe der Zeit ändern, ist es üblich, dass sich auch ILM-Richtlinien im Laufe der Zeit ändern müssen. Das Ändern von ILM-Richtlinien ist eine zentrale Funktion, mit der sich StorageGRID-Kunden schnell und einfach an die sich ständig verändernde Umgebung anpassen können. Bitte prüfen Sie das "ILM-Richtlinie" Und "ILM-Regeln" Einrichtung in StorageGRID.

Leistung

StorageGRID skaliert die Performance, indem weitere Storage-Nodes hinzugefügt werden. Dabei können es sich um VMs, Bare Metal oder speziell entwickelte Appliances wie die handelt "SG5712, SG5760, SG6060 ODER SGF6024". In unseren Tests haben wir mit der SGF6024 Appliance die wesentlichen Performance-Anforderungen von Apache Kafka übertroffen. Das Grid mit drei Nodes war eine minimale Größe. Wenn Kunden ihren Kafka Cluster mit zusätzlichen Vermittlern skalieren, können sie zur Steigerung von Performance und Kapazität weitere Storage-Nodes hinzufügen.

Load Balancer- und Endpunkt-Konfiguration

Admin-Nodes in StorageGRID stellen die Grid Manager-Benutzeroberfläche (Benutzeroberfläche) und DEN REST-API-Endpunkt zum Anzeigen, Konfigurieren und Managen des StorageGRID-Systems sowie Audit-Protokolle zur Nachverfolgung der Systemaktivitäten bereit. Um einen hochverfügbaren S3-Endpunkt für Conflient Kafka Tiered Storage zu bieten, haben wir den StorageGRID Load Balancer implementiert, der als Service auf Admin-Nodes und Gateway-Nodes ausgeführt wird. Darüber hinaus managt der Load Balancer auch den lokalen Datenverkehr und spricht mit dem GSLB (Global Server Load Balancing), um die Disaster Recovery zu unterstützen.

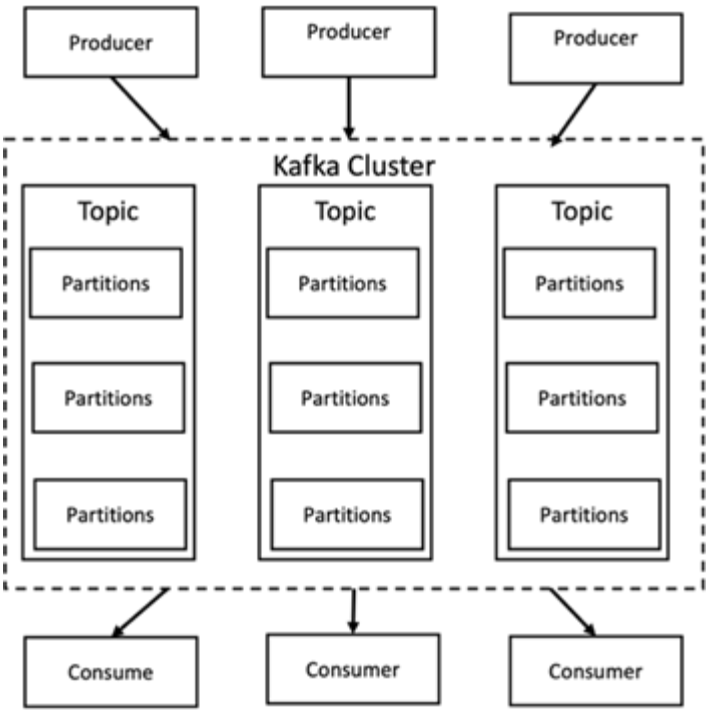
Zur weiteren Verbesserung der Endpoint-Konfiguration bietet StorageGRID Richtlinien für die Traffic-Klassifizierung, die in den Admin-Node integriert sind, mit denen Sie Ihren Workload-Datenverkehr überwachen und verschiedene Quality of Service (QoS)-Einschränkungen für Ihre Workloads anwenden können. Richtlinien zur Traffic-Klassifizierung werden auf Endpunkte im StorageGRID Load Balancer Service für Gateway-Nodes und Admin-Nodes angewendet. Diese Richtlinien können bei der Gestaltung und Überwachung des Verkehrs helfen.

Verkehrsklassifizierung in StorageGRID

StorageGRID verfügt über integrierte QoS-Funktionen. Richtlinien zur Traffic-Klassifizierung unterstützen das Monitoring verschiedener Arten von S3 Traffic, der von einer Client-Applikation stammt. Anschließend können Sie Richtlinien erstellen und anwenden, um Höchstwerte für diesen Datenverkehr festzulegen. Diese basieren auf der ein-/Ausfahrbandbreite, der Anzahl der gleichzeitigen Lese-/Schreibanfragen oder der Anfraquote für Lese- und Schreibvorgänge.

Apache Kafka

Apache Kafka ist eine Framework-Implementierung eines Software-Busses mit Stream Processing geschrieben in Java und Scala. Das Ziel ist eine einheitliche Plattform mit hohem Durchsatz und niedriger Latenz für die Verarbeitung von Echtzeit-Datenfeeds. Kafka kann sich mit einem externen System für den Datenexport und den Import über Kafka Connect verbinden und Kafka Streams, eine Java Stream Processing Library, zur Verfügung stellen. Kafka verwendet ein binäres, TCP-basiertes Protokoll, das für Effizienz optimiert ist und auf einer Abstraktion basiert, die Nachrichten automatisch gruppiert, um den Overhead von NetzwerkRoundtrip zu reduzieren. Dies ermöglicht größere sequenzielle Festplattenoperationen, größere Netzwerkpakete und zusammenhängende Speicherblöcke. Kafka kann also einen sprunghafte Strom zufälliger Nachrichtenschreibvorgänge in lineare Schreibvorgänge verwandeln. Die folgende Abbildung zeigt den grundlegenden Datenfluss von Apache Kafka.



Kafka speichert Schlüsselwertbotschaften, die aus einer willkürlichen Anzahl von Prozessen stammen, die Produzenten genannt werden. Die Daten lassen sich innerhalb verschiedener Themen in unterschiedliche Partitionen unterpartitionieren. Innerhalb einer Partition werden Nachrichten streng nach ihren Offsets (der Position einer Nachricht innerhalb einer Partition) geordnet und zusammen mit einem Zeitstempel indiziert und gespeichert. Andere Prozesse, die Verbraucher genannt werden, können Nachrichten von Partitionen lesen. Zur Stream-Verarbeitung bietet Kafka die Streaming-API, mit der Java-Applikationen geschrieben werden können, die Daten aus Kafka verbrauchen und Ergebnisse an Kafka schreiben. Apache Kafka arbeitet auch mit externen Streamverarbeitungssystemen wie Apache Apex, Apache Flink, Apache Spark, Apache Storm und Apache NiFi zusammen.

Kafka läuft auf einem Cluster aus einem oder mehreren Servern (sogenannten Brokern) und die Partitionen aller Themen sind über die Cluster Nodes verteilt. Darüber hinaus werden Partitionen zu mehreren Brokern repliziert. Diese Architektur ermöglicht es Kafka, massive Nachrichten-Streams fehlertolerant zu liefern und hat es ermöglicht, einige der herkömmlichen Messaging-Systeme wie Java Message Service (JMS), Advanced Message Queuing Protocol (AMQP) usw. zu ersetzen. Seit Version 0.11.0.0 bietet Kafka transaktionale Schreibvorgänge, die mithilfe der Streaming-API exakt einmal Stream-Verarbeitung sorgen.

Kafka unterstützt zwei Themenarten: Regelmäßig und verdichtet. Normale Themen können mit einer Aufbewahrungszeit oder mit einer Speicherplatzbindung konfiguriert werden. Wenn Datensätze älter als die angegebene Aufbewahrungszeit sind oder wenn der gebundene Speicherplatz für eine Partition überschritten wird, kann Kafka alte Daten in freien Speicherplatz löschen. Standardmäßig werden Themen mit einer Aufbewahrungszeit von 7 Tagen konfiguriert, es ist aber auch möglich, Daten unbegrenzt zu speichern. Bei verdichteten Themen verfallen Datensätze aufgrund von Zeit- oder Raumgrenzen nicht. Stattdessen behandelt Kafka spätere Nachrichten als Aktualisierungen älterer Nachrichten mit dem gleichen Schlüssel und garantiert, dass sie nie die neueste Nachricht pro Schlüssel löschen. Benutzer können Nachrichten vollständig löschen, indem sie eine sogenannte Tombstone-Nachricht mit dem Null-Wert für einen bestimmten Schlüssel schreiben.

In Kafka gibt es fünf große APIs:

- **Producer API.** erlaubt eine Anwendung, Streams von Datensätzen zu veröffentlichen.
- **Consumer API.** ermöglicht eine Anwendung, Themen zu abonnieren und Datenströme zu verarbeiten.
- **Connector API.** führt die wiederverwendbaren Producer- und Consumer-APIs aus, die die Themen mit

den vorhandenen Anwendungen verknüpfen können.

- **Streams API.** Diese API wandelt die Input Streams in Output um und erzeugt das Ergebnis.
- **Admin API.** zur Verwaltung von Kafka-Themen, Brokern und anderen Kafka-Objekten.

Die Consumer and Producer APIs bauen auf dem Kafka Messaging-Protokoll auf und bieten eine Referenzimplementierung für Kafka-Verbraucher und -Produzenten in Java an. Das zugrunde liegende Messaging-Protokoll ist ein binäres Protokoll, mit dem Entwickler ihre eigenen Verbraucher- oder Producer-Clients in jeder Programmiersprache schreiben können. Damit erschließt sich Kafka aus dem Java Virtual Machine (JVM) Ecosystem. Eine Liste der nicht-Java-Clients wird im Apache Kafka Wiki gepflegt.

Anwendungsfälle für Apache Kafka

Apache Kafka ist besonders beliebt bei Messaging, Website-Aktivitäten-Tracking, Metriken, Log-Aggregation, Stream Processing, Event Sourcing und Protokollierung übergeben.

- Kafka bietet einen verbesserten Durchsatz, integrierte Partitionierung, Replizierung und Fehlertoleranz und ist somit eine gute Lösung für große Applikationen zur Nachrichtenverarbeitung.
- Kafka kann die Aktivitäten eines Benutzers (Seitenaufrufe und Suchen) in einer Pipeline für die Nachverfolgung als Set von Veröffentlichungsdaten in Echtzeit neu erstellen.
- Kafka wird häufig für Daten aus betrieblichen Monitoring eingesetzt. Dazu gehört die Zusammenfassung von Statistiken aus verteilten Applikationen zur Erstellung zentralisierter Feeds von Betriebsdaten.
- Viele Anwender verwenden Kafka als Ersatz für eine Log-Aggregationslösung. Die Log-Aggregation sammelt üblicherweise physische Log-Dateien von den Servern und stellt sie zur Verarbeitung an einem zentralen Ort (z. B. einem Dateiserver oder HDFS). Kafka abstrahiert Dateidetails und ermöglicht eine saubere Abstraktion von Protokoll- oder Ereignisdaten als Nachrichtenstrom. Die Verarbeitung mit niedriger Latenz wird vereinfacht, es werden diverse Datenquellen und verteilte Datennutzung unterstützt.
- Viele Anwender von Kafka verarbeiten Daten in mehreren Etappen, in denen aus Kafka-Themen Rohdaten gesammelt und dann aggregiert, angereichert oder anderweitig in neue Themen umgewandelt werden, um sie weiter zu nutzen oder nachbearbeiten zu können. So könnte beispielsweise eine Verarbeitungspipeline für die Empfehlung von Nachrichtenartikeln Artikelinhalte aus RSS-Feeds kriechen und in ein "Artikel"-Thema veröffentlichen. Eine weitere Verarbeitung könnte diesen Inhalt normalisieren oder deduplizieren und den bereinigten Artikelinhalt in einem neuen Thema veröffentlichen. In einer letzten Phase der Verarbeitung könnte möglicherweise versucht werden, diesen Inhalt an Anwender zu empfehlen. Solche Verarbeitungspipelines erstellen Grafiken von Echtzeit-Datenströmen auf Basis der einzelnen Themen.
- Event Souring ist eine Art Anwendungsdesign, bei der Zustandsänderungen als eine zeitgeordnete Sequenz von Datensätzen protokolliert werden. Da Kafka sehr große gespeicherte Protokolldaten unterstützt, eignet es sich hervorragend als Back-End für eine Anwendung dieser Art.
- Kafka kann eine Art externes Commit-Log für ein verteiltes System dienen. Das Protokoll hilft beim Replizieren von Daten zwischen Nodes und dient als Mechanismus zur Neusynchronisierung zur Wiederherstellung fehlgeschlagener Nodes. Die Log-Data-Compaction-Funktion in Kafka unterstützt diesen Anwendungsfall.

Fließend

Conflient Platform ist eine Plattform für Unternehmen, die Kafka mit fortschrittlichen Funktionen abrundet, die dazu dienen, die Applikationsentwicklung und -Konnektivität zu beschleunigen, Transformationen durch Stream-Verarbeitung zu ermöglichen, skalierbare Enterprise-Prozesse zu vereinfachen und anspruchsvolle Architekturanforderungen zu erfüllen. Confluent wurde von den ursprünglichen Schöpfern von Apache Kafka erbaut und erweitert die Vorteile von Kafka mit Funktionen der Enterprise-Klasse, ohne Kafka-Management oder -Monitoring zu belasten. Heute sind mehr als 80 % der Fortune 100-Unternehmen auf Data-Streaming-Technologie gestützt – und die meisten von ihnen nutzen Confluent.

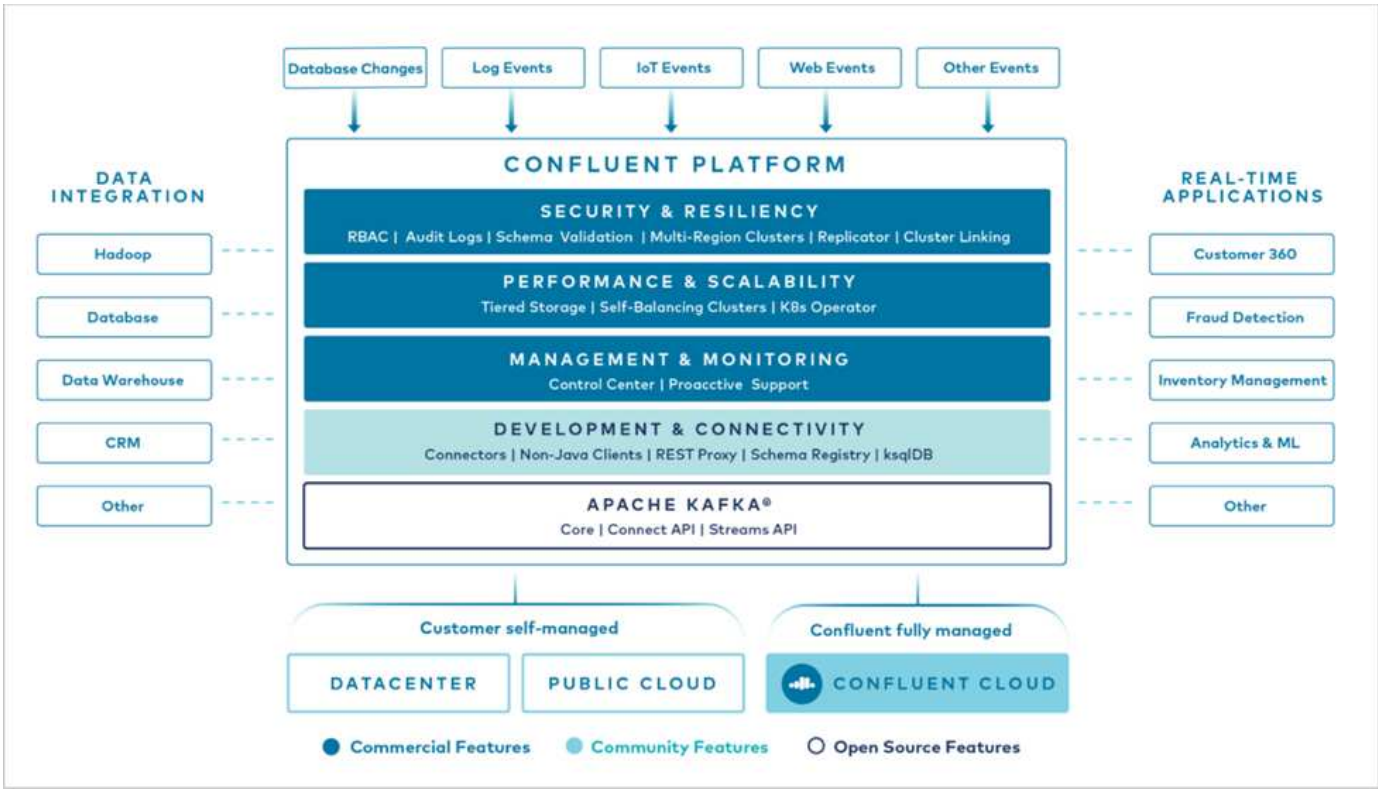
Warum Confluent?

Durch die Integration von historischen und Echtzeit-Daten in eine einzige, zentrale Quelle der Wahrheit erleichtert Confluent den Aufbau einer völlig neuen Kategorie moderner, ereignisgesteuerter Anwendungen, die Erstellung einer universellen Datenpipeline und die Nutzung leistungsstarker neuer Anwendungsfälle mit voller Skalierbarkeit, Leistung und Zuverlässigkeit.

Wofür wird Confluent verwendet?

Mit der Conflient Platform können Sie sich darauf konzentrieren, wie Sie aus Ihren Daten einen geschäftlichen Nutzen ziehen können, statt sich um die zugrunde liegenden Mechanismen sorgen zu müssen, wie beispielsweise der Transport oder die Integration von Daten zwischen verschiedenen Systemen. Confluent Platform vereinfacht insbesondere die Anbindung von Datenquellen an Kafka, die Erstellung von Streaming-Applikationen sowie die Sicherung, Überwachung und das Management der Kafka Infrastruktur. Heute wird Confluent Platform für eine Vielzahl von Anwendungsbeispielen in zahlreichen Branchen eingesetzt, von Finanzdienstleistungen über Omnichannel-Einzelhandel und autonome Fahrzeuge bis hin zur Betrugserkennung, Microservices und IoT.

Die folgende Abbildung zeigt die Komponenten der Conflient Kafka Platform.



Überblick über die Event-Streaming-Technologie von Confluent

Der Kern der Confluent Platform ist "Apache Kafka", Die beliebteste verteilte Open-Source-Streaming-Plattform. Kafka bietet folgende wichtige Funktionen:

- Veröffentlichen und abonnieren Sie Datenströme.
- Fehlertolerante Speicherung von Datenströmen
- Verarbeiten von Datensätzen.

Die Confluent Platform umfasst außerdem Schema Registry, REST Proxy, insgesamt 100+ vordefinierte Kafka-

Anschlüsse und ksqlDB.

Überblick über die Enterprise-Funktionen der Confluent Plattform

- **Confluent Control Center.** Ein GUI-basiertes System zur Verwaltung und Überwachung von Kafka. Damit können Sie Kafka Connect ganz einfach verwalten und Verbindungen zu anderen Systemen erstellen, bearbeiten und verwalten.
- **Fließend für Kubernetes.** der fließende für Kubernetes ist ein Kubernetes Operator. Kubernetes-Betreiber erweitern die Orchestrierungsfunktionen von Kubernetes um spezielle Funktionen und Anforderungen für eine spezifische Plattform-Applikation. Bei Confluent Platform müssen dazu die Implementierung von Kafka auf Kubernetes erheblich vereinfacht und typische Aufgaben im Infrastruktur-Lebenszyklus automatisiert werden.
- **Confluent Connectors to Kafka.** Connectors verwenden die Kafka Connect API, um Kafka mit anderen Systemen wie Datenbanken, Schlüsselwertspeicher, Suchindizes und Dateisystemen zu verbinden. Confluent Hub verfügt über herunterladbare Anschlüsse für die beliebtesten Datenquellen und Waschbecken, einschließlich vollständig getestete und unterstützte Versionen dieser Anschlüsse mit Confluent Platform. Weitere Details finden Sie hier "Hier".
- **Self-Balancing Cluster.** bietet automatisches Load Balancing, Fehlererkennung und Selbstheilung. Broker können nach Bedarf und ohne manuelles Tuning hinzugefügt oder ausmustern.
- **Fließende Cluster-Verknüpfung.** verbindet Cluster direkt miteinander und spiegelt Themen von einem Cluster zum anderen über eine Link-Bridge. Die Cluster-Verknüpfung vereinfacht die Einrichtung von Implementierungen mit mehreren Rechenzentren, mehreren Clustern und Hybrid Clouds.
- **Confluent Auto Data Balancer.** überwacht Ihren Cluster für die Anzahl der Broker, die Größe der Partitionen, Anzahl der Partitionen und die Anzahl der Führer innerhalb des Clusters. Auf diese Weise können Sie Daten verschieben, um einen geraden Workload über Ihr Cluster zu erstellen, und gleichzeitig den Datenverkehr neu verteilen, um die Auswirkungen auf die Produktions-Workloads bei der Ausbalancierung zu minimieren.
- **Confluent Replikator.** macht es einfacher als je zuvor, mehrere Kafka Cluster in mehreren Rechenzentren zu pflegen.
- **Tiered Storage.** bietet Optionen zur Speicherung großer Kafka-Datenmengen mit Ihrem bevorzugten Cloud-Provider und reduziert so die Betriebskosten und die Kosten. Mit Tiered Storage können Sie Daten auf kostengünstigem Objekt-Storage und Vermittlern nur dann aufbewahren, wenn Sie mehr Computing-Ressourcen benötigen.
- **Confluent JMS Client.** Confluent Platform enthält einen JMS-kompatiblen Client für Kafka. Dieser Kafka-Client implementiert die JMS 1.1 Standard-API und verwendet Kafka-Broker als Backend. Dies ist nützlich, wenn vorhandene Anwendungen JMS verwenden und Sie den vorhandenen JMS-Nachrichten-Broker durch Kafka ersetzen möchten.
- **Confluent MQTT Proxy.** bietet eine Möglichkeit, Daten direkt an Kafka von MQTT-Geräten und Gateways zu veröffentlichen, ohne dass ein MQTT-Broker in der Mitte nötig ist.
- **Confluent Security Plugins.** Confluent Security Plugins werden verwendet, um Sicherheitsfunktionen zu verschiedenen Tools und Produkten der Confluent Platform hinzuzufügen. Derzeit gibt es ein Plugin für den Confluent REST Proxy, das hilft, die eingehenden Anfragen zu authentifizieren und den authentifizierten Principal an Anfragen an Kafka zu verbreiten. Auf diese Weise können Confluent REST Proxy-Clients die mandantenfähigen Sicherheitsfunktionen des Kafka-Brokers nutzen.

"Als Nächstes: Confluent Verification."

Confluent Verification

"Zurück: Technologieübersicht."

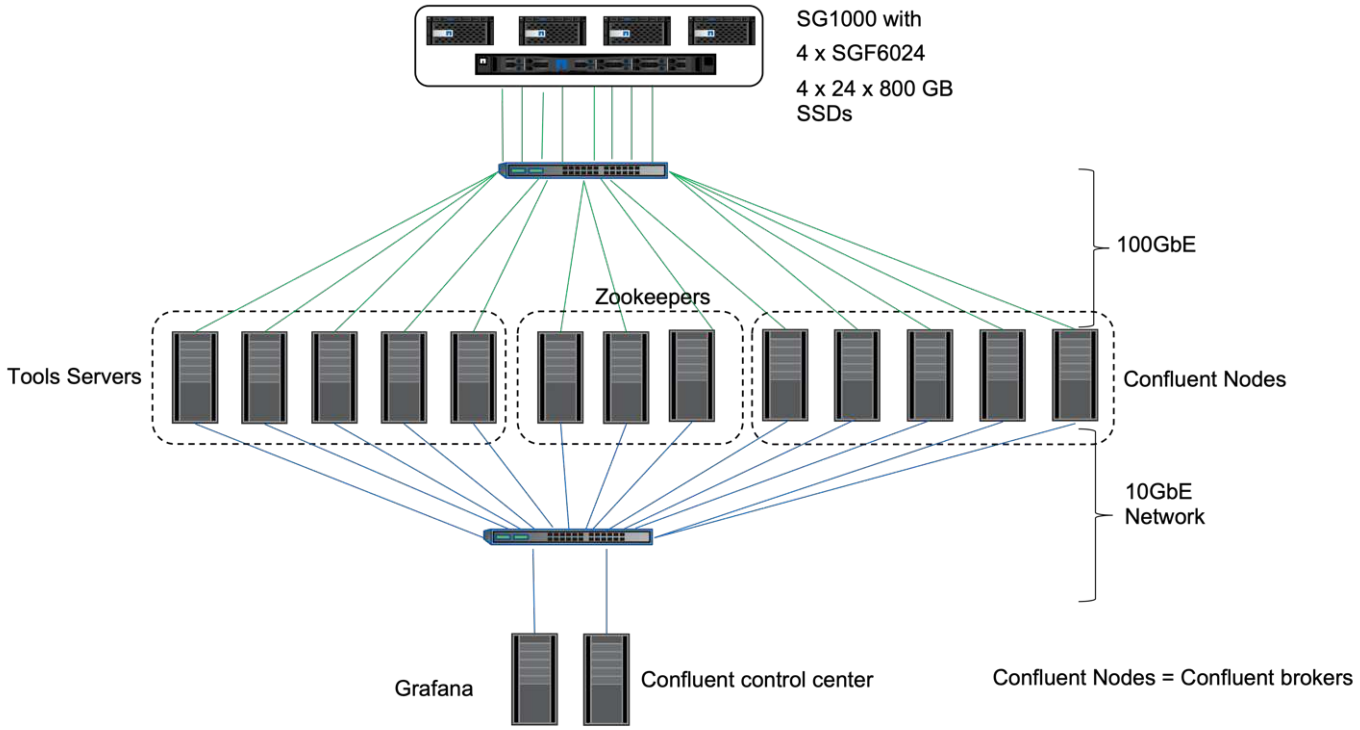
Wir haben eine Überprüfung mit Confluent Platform 6.2 Tiered Storage in NetApp StorageGRID durchgeführt. Die NetApp und Conflient Teams haben diese Verifizierung gemeinsam durchgeführt und die für die Verifizierung erforderlichen Testfälle durchgeführt.

Plattformeinrichtung fließend

Zur Überprüfung wurde folgendes Setup verwendet.

Zur Überprüfung wurden drei Zookeeper, fünf Broker, fünf Testskripte mit Servern, benannte Tools mit 256 GB RAM und 16 CPUs verwendet. Für NetApp Storage haben wir StorageGRID mit einem SG1000 Load Balancer mit vier SGF6024s verwendet. Der Storage und die Broker waren über 100-GbE-Verbindungen verbunden.

Die folgende Abbildung zeigt die Netzwerktopologie der Konfiguration, die für die Confluent-Überprüfung verwendet wird.



Die Tools-Server fungieren als Anwendungsclients, die Anfragen an Confluent Nodes senden.

Fließende Tiered Storage-Konfiguration

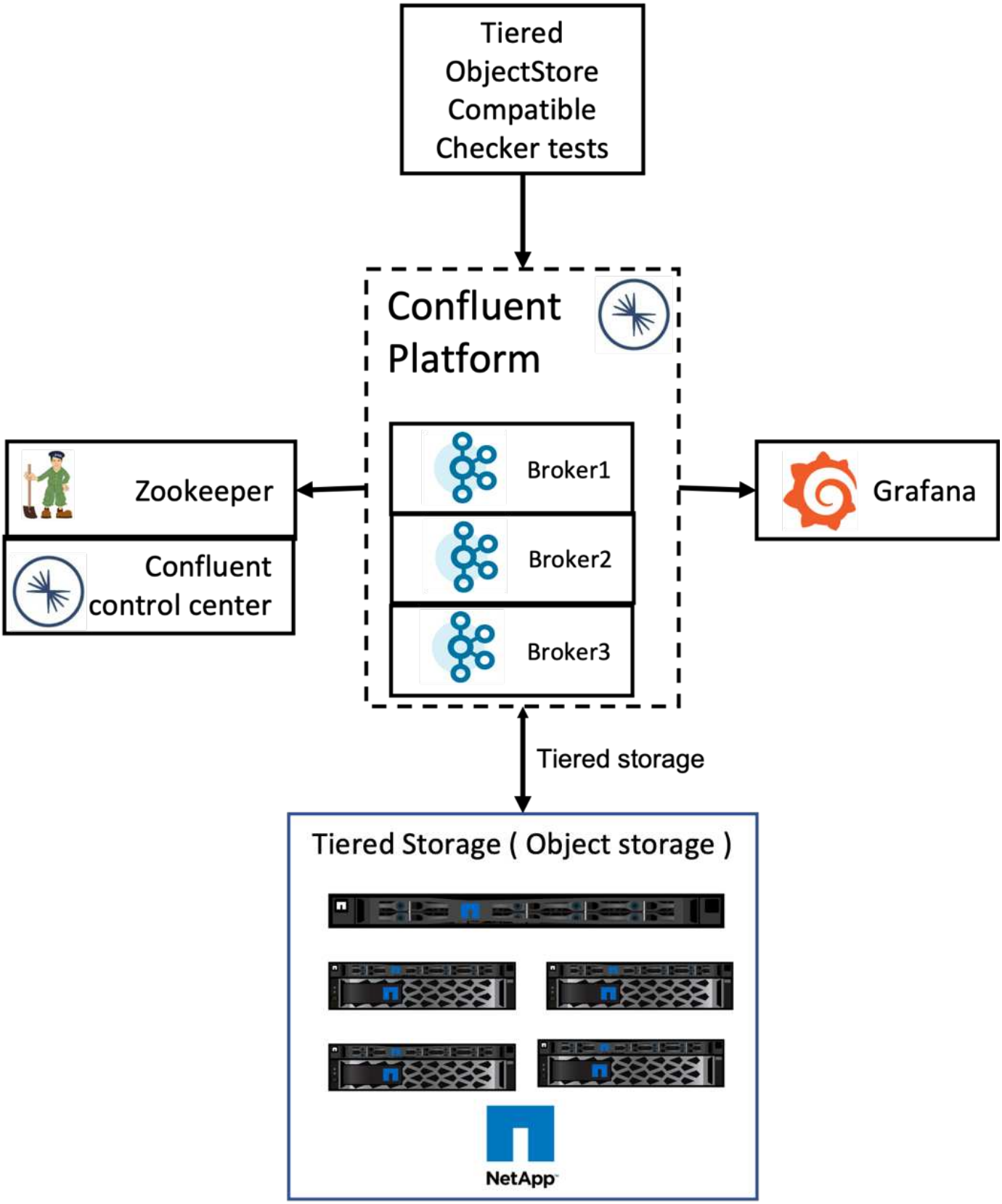
Für die Tiered Storage-Konfiguration sind in Kafka die folgenden Parameter erforderlich:

```
Confluent.tier.archiver.num.threads=16
confluent.tier.fetcher.num.threads=32
confluent.tier.enable=true
confluent.tier.feature=true
confluent.tier.backend=S3
confluent.tier.s3.bucket=kafkasgdbucket1-2
confluent.tier.s3.region=us-west-2
confluent.tier.s3.cred.file.path=/data/kafka/.ssh/credentials
confluent.tier.s3.aws.endpoint.override=http://kafkasgd.rtpppe.netapp.com:
10444/
confluent.tier.s3.force.path.style.access=true
```

Zur Überprüfung wurde StorageGRID mit dem HTTP-Protokoll verwendet, aber HTTPS funktioniert auch. Der Zugriffsschlüssel und der Geheimschlüssel werden im Dateinamen im gespeichert `confluent.tier.s3.cred.file.path` Parameter.

NetApp Objekt-Storage – StorageGRID

Wir haben Single-Site-Konfiguration in StorageGRID für die Verifikation konfiguriert.



Verifizierungstests

Wir haben die folgenden fünf Testfälle für die Verifizierung abgeschlossen. Diese Tests werden im Trogdor-Rahmen durchgeführt. Die ersten beiden waren Funktionstests und die verbleibenden drei waren Performance-Tests.

Korrektheit des Objektspeichers

Dieser Test bestimmt, ob alle grundlegenden Vorgänge (zum Beispiel get/put/delete) auf der Objektspeicher-API gut gemäß den Anforderungen an Tiered Storage funktionieren. Es ist ein grundlegender Test, dass jeder Objektspeicher-Service den folgenden Tests voraus sein sollte. Es handelt sich um einen durchsetzungsfähigen Test, der entweder besteht oder ausfällt.

Funktionstest der Tiering-Funktion

Dieser Test prüft, ob die End-to-End-Tiered Storage-Funktionalität einem durchsetzungsfähigen Test gut funktioniert, der entweder erfolgreich oder ausfallen sollte. Der Test erstellt ein Testthema, das standardmäßig mit aktiviertem Tiering und einer stark reduzierten Hotset-Größe konfiguriert ist. Es erzeugt einen Ereignisstrom zum neu erstellten Testthema, wartet darauf, dass die Broker die Segmente im Objektspeicher archivieren und dann den Ereignisstrom nutzt und überprüft, ob der verbrauchte Strom mit dem erzeugten Stream übereinstimmt. Die Anzahl der Nachrichten, die an den Ereignisstrom erzeugt werden, ist konfigurierbar, sodass der Benutzer je nach Testanforderungen eine ausreichend große Arbeitslast generieren kann. Die reduzierte Hotset-Größe stellt sicher, dass der Verbraucher außerhalb des aktiven Segments nur aus dem Objektspeicher bedient wird; dies hilft, die Richtigkeit des Objektspeichers auf Lesevorgänge zu prüfen. Diesen Test haben wir mit und ohne Objektspeicherfehlereinspritzung durchgeführt. Wir haben einen Node-Ausfall simuliert, indem wir den Service Manager Service in einem der Nodes in StorageGRID unterbrechen und validieren, dass die End-to-End-Funktionalität mit dem Objekt-Storage kompatibel ist.

Tier-Fetch-Benchmark

Dieser Test bestätigte die Lese-Performance des Tiered Objekt-Storage und prüfte den Bereich, in dem Leseanforderungen unter hoher Last aus vom Benchmark erzeugten Segmenten abgerufen werden. In dieser Benchmark hat Confluent benutzerdefinierte Clients entwickelt, um die Tier-fetch-Anforderungen zu erfüllen.

Erstellen von Benchmark-Workloads für die Nutzung

Dieser Test erzeugte über die Archivierung von Segmenten einen indirekten Schreib-Workload auf dem Objektspeicher. Der Lese-Workload (Segmente Lesen) wurde aus dem Objekt-Storage generiert, wenn Verbrauchergruppen die Segmente abgerufen haben. Dieser Workload wurde durch das Testskript generiert. In diesem Test wurde die Performance von Lese- und Schreibvorgängen im Objekt-Storage in parallelen Threads überprüft. Wir haben wie bei dem Tiering-Funktionstest mit und ohne Objektspeicher-Fehlereinspritzung getestet.

Benchmarks für Retention Workloads

Bei diesem Test wurde die Löschleistung eines Objektspeichers unter einem hohen Workload für die Aufbewahrung von Themen überprüft. Der Retention Workload wurde mit einem Testskript generiert, das parallel zu einem Testthema viele Nachrichten erzeugt. Das Testthema wurde mit einer aggressiven größenbasierten und zeitbasierten Aufbewahrungseinstellung konfiguriert, die dazu führte, dass der Ereignisstrom kontinuierlich aus dem Objektspeicher gelöscht wurde. Die Segmente wurden anschließend archiviert. Dies führte zu einer großen Anzahl von Löschungen im Objekt-Storage durch den Vermittler und die Sammlung der Performance bei Löschvorgängen im Objektspeicher.

"Weiter: Performance-Tests mit Skalierbarkeit."

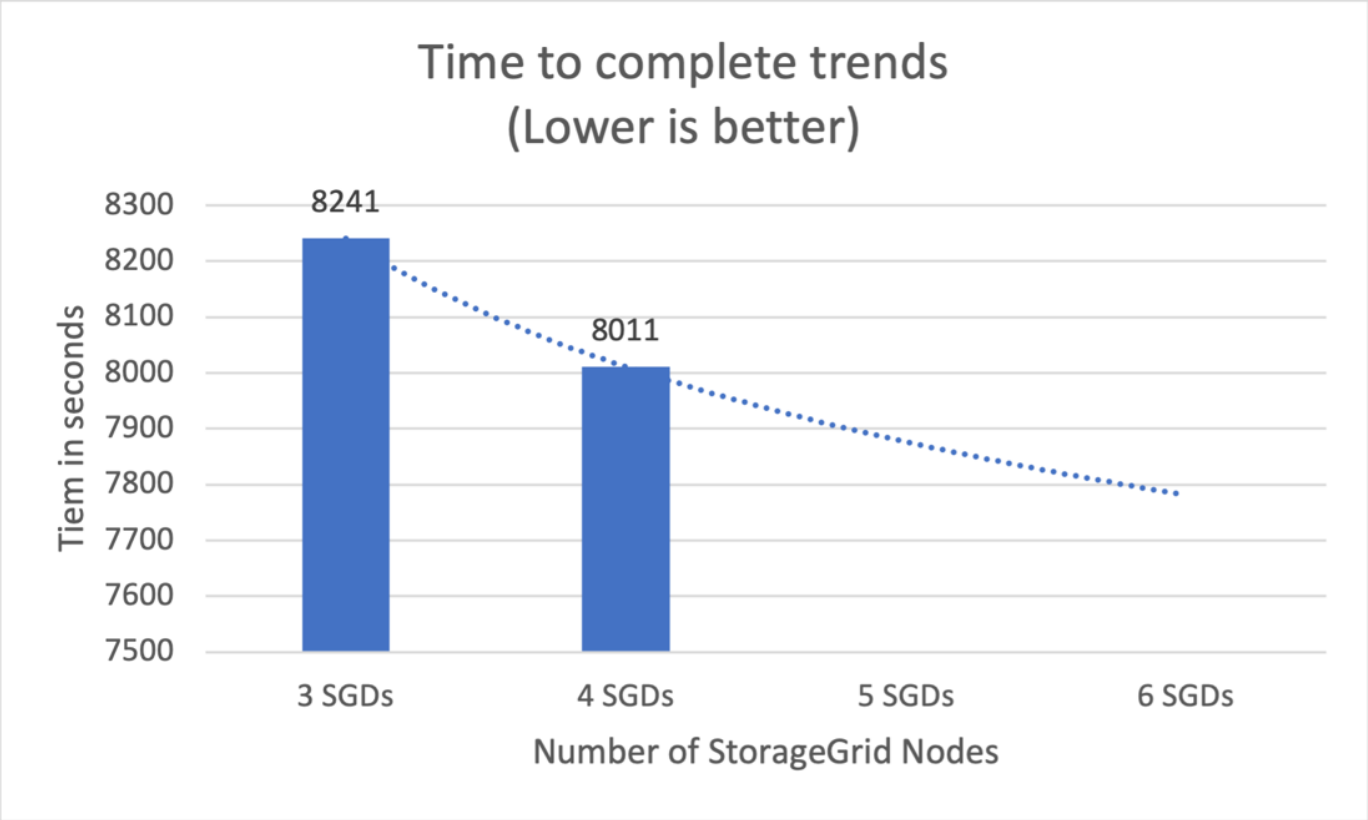
Performance-Tests mit Skalierbarkeit

"Früher: Confluent Überprüfung."

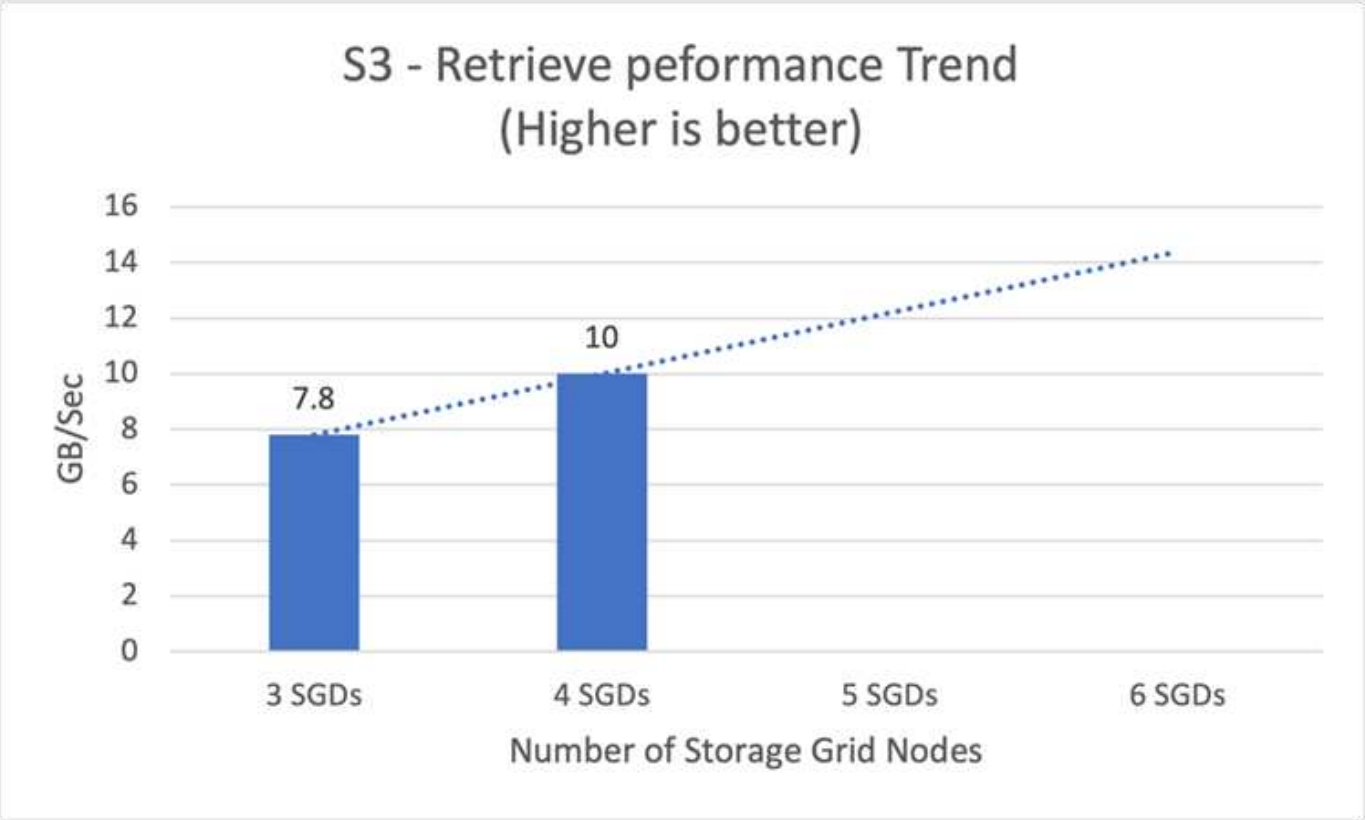
Wir haben die Tiered Storage-Tests mit drei bis vier Nodes für Producer und Verbraucher-Workloads mithilfe

des NetApp StorageGRID Setups durchgeführt. Laut unseren Tests waren die Fertigstellungszeit und die Performance-Ergebnisse direkt proportional zur Anzahl der StorageGRID Nodes. Das StorageGRID-Setup benötigte mindestens drei Nodes.

- Die Zeit bis zur Fertigstellung des Produce- und Consumer-Vorgangs sank linear, wenn die Anzahl der Storage Nodes gestiegen ist.



- Die Performance für den s3-Abrufvorgang ist basierend auf der Anzahl der StorageGRID-Nodes linear gestiegen. StorageGRID unterstützt bis zu 200 StorageGRID Nodes.



"Weiter: Anschluss mit fließenden s3."

Fließender s3-Anschluss

"Früher: Performance-Tests mit Skalierbarkeit."

Der Amazon S3 Sink Connector exportiert Daten von Apache Kafka Themen entweder im Avro-, JSON- oder Byte-Format in S3-Objekte. Der Amazon S3 Sink Connector fragt regelmäßig Daten aus Kafka ab und lädt sie wiederum nach S3 hoch. Ein Partitioner wird verwendet, um die Daten jeder Kafka-Partition in Stücke zu teilen. Jeder Datenblock wird als S3-Objekt dargestellt. Der Schlüsselname kodiert das Thema, die Kafka-Partition und den Start-Offset dieses Datenblocks.

In diesen Einstellungen zeigen wir Ihnen, wie man Themen aus Kafka direkt über den Kafka s3 Waschbecken im Objekt-Storage liest und schreibt. Für diesen Test haben wir einen eigenständigen Confluent Cluster verwendet, dieses Setup gilt jedoch für einen verteilten Cluster.

1. Laden Sie Confluent Kafka von der Confluent Website herunter.
2. Packen Sie das Paket in einen Ordner auf Ihrem Server aus.
3. Exportieren Sie zwei Variablen.

```
Export CONFLUENT_HOME=/data/confluent/confluent-6.2.0
export PATH=$PATH:/data/confluent/confluent-6.2.0/bin
```

4. Für ein eigenständiges Confluent Kafka Setup erstellt der Cluster einen temporären Stammordner in `/tmp`. Es erstellt auch Zookeeper, Kafka, eine Schemaregistrierung, connect, einen ksql-Server, Control-Center Ordner und kopiert ihre jeweiligen Konfigurationsdateien von `$CONFLUENT_HOME`. Das folgende Beispiel

zeigt:

```
root@stlrx2540m1-108:~# ls -ltr /tmp/confluent.406980/
total 28
drwxr-xr-x 4 root root 4096 Oct 29 19:01 zookeeper
drwxr-xr-x 4 root root 4096 Oct 29 19:37 kafka
drwxr-xr-x 4 root root 4096 Oct 29 19:40 schema-registry
drwxr-xr-x 4 root root 4096 Oct 29 19:45 kafka-rest
drwxr-xr-x 4 root root 4096 Oct 29 19:47 connect
drwxr-xr-x 4 root root 4096 Oct 29 19:48 ksql-server
drwxr-xr-x 4 root root 4096 Oct 29 19:53 control-center
root@stlrx2540m1-108:~#
```

5. Zookeeper Konfigurieren. Sie müssen nichts ändern, wenn Sie die Standardparameter verwenden.

```
root@stlrx2540m1-108:~# cat
/tmp/confluent.406980/zookeeper/zookeeper.properties  | grep -iv ^#
dataDir=/tmp/confluent.406980/zookeeper/data
clientPort=2181
maxClientCnxns=0
admin.enableServer=false
tickTime=2000
initLimit=5
syncLimit=2
server.179=controlcenter:2888:3888
root@stlrx2540m1-108:~#
```

In der obigen Konfiguration haben wir die aktualisiert `server. xxx` Eigenschaft. Standardmäßig benötigen Sie drei Zookeeper für die Auswahl von Kafka Leader.

6. Wir haben eine myid-Datei in erstellt `/tmp/confluent.406980/zookeeper/data` Mit einer eindeutigen ID:

```
root@stlrx2540m1-108:~# cat /tmp/confluent.406980/zookeeper/data/myid
179
root@stlrx2540m1-108:~#
```

Wir haben die letzte Anzahl von IP-Adressen für die myid-Datei verwendet. Für Kafka wurden Standardwerte verwendet: Connect, Control Center, Kafka, Kafka-Rest, Konfiguration von ksql-Server und Schema-Registry

7. Starten Sie die Kafka-Dienste.

```
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# confluent
local services  start
The local commands are intended for a single-node development
environment only,
NOT for production usage.

Using CONFLUENT_CURRENT: /tmp/confluent.406980
ZooKeeper is [UP]
Kafka is [UP]
Schema Registry is [UP]
Kafka REST is [UP]
Connect is [UP]
ksqlDB Server is [UP]
Control Center is [UP]
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

Für jede Konfiguration gibt es einen Protokollordner, der die Fehlerbehebung erleichtert. In einigen Fällen nehmen Services mehr Zeit zum Start in Anspruch. Stellen Sie sicher, dass alle Services betriebsbereit sind.

8. Installieren Sie Kafka Connect mit `confluent-hub`.

```
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# ./confluent-
hub install confluentinc/kafka-connect-s3:latest
The component can be installed in any of the following Confluent
Platform installations:
  1. /data/confluent/confluent-6.2.0 (based on $CONFLUENT_HOME)
  2. /data/confluent/confluent-6.2.0 (where this tool is installed)
Choose one of these to continue the installation (1-2): 1
Do you want to install this into /data/confluent/confluent-
6.2.0/share/confluent-hub-components? (yN) y

Component's license:
Confluent Community License
http://www.confluent.io/confluent-community-license
I agree to the software license agreement (yN) y
Downloading component Kafka Connect S3 10.0.3, provided by Confluent,
Inc. from Confluent Hub and installing into /data/confluent/confluent-
6.2.0/share/confluent-hub-components
Do you want to uninstall existing version 10.0.3? (yN) y
Detected Worker's configs:
  1. Standard: /data/confluent/confluent-6.2.0/etc/kafka/connect-
distributed.properties
  2. Standard: /data/confluent/confluent-6.2.0/etc/kafka/connect-
standalone.properties
```

```
  3. Standard: /data/confluent/confluent-6.2.0/etc/schema-
registry/connect-avro-distributed.properties
  4. Standard: /data/confluent/confluent-6.2.0/etc/schema-
registry/connect-avro-standalone.properties
  5. Based on CONFLUENT_CURRENT:
/tmp/confluent.406980/connect/connect.properties
  6. Used by Connect process with PID 15904:
/tmp/confluent.406980/connect/connect.properties
Do you want to update all detected configs? (yN) y
Adding installation directory to plugin path in the following files:
  /data/confluent/confluent-6.2.0/etc/kafka/connect-
distributed.properties
  /data/confluent/confluent-6.2.0/etc/kafka/connect-
standalone.properties
  /data/confluent/confluent-6.2.0/etc/schema-registry/connect-avro-
distributed.properties
  /data/confluent/confluent-6.2.0/etc/schema-registry/connect-avro-
standalone.properties
  /tmp/confluent.406980/connect/connect.properties
  /tmp/confluent.406980/connect/connect.properties

Completed
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

Sie können eine bestimmte Version auch mithilfe von installieren `confluent-hub install confluentinc/kafka-connect-s3:10.0.3`.

9. Standardmäßig ist `confluentinc-kafka-connect-s3` Ist in installiert `/data/confluent/confluent-6.2.0/share/confluent-hub-components/confluentinc-kafka-connect-s3`.
10. Aktualisieren Sie den Plug-in-Pfad mit dem neuen `confluentinc-kafka-connect-s3`.

```
root@stlrx2540m1-108:~# cat /data/confluent/confluent-
6.2.0/etc/kafka/connect-distributed.properties | grep plugin.path
#
plugin.path=/usr/local/share/java,/usr/local/share/kafka/plugins,/opt/co
nnectors,
plugin.path=/usr/share/java,/data/zookeeper/confluent/confluent-
6.2.0/share/confluent-hub-components,/data/confluent/confluent-
6.2.0/share/confluent-hub-components,/data/confluent/confluent-
6.2.0/share/confluent-hub-components/confluentinc-kafka-connect-s3
root@stlrx2540m1-108:~#
```

11. Beenden Sie die Confluent-Dienste, und starten Sie sie neu.

```
confluent local services  stop
confluent local services  start
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# confluent
local services  status
The local commands are intended for a single-node development
environment only,
NOT for production usage.

Using CONFLUENT_CURRENT: /tmp/confluent.406980
Connect is [UP]
Control Center is [UP]
Kafka is [UP]
Kafka REST is [UP]
ksqlDB Server is [UP]
Schema Registry is [UP]
ZooKeeper is [UP]
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

12. Konfigurieren Sie die Zugriffs-ID und den geheimen Schlüssel im `/root/.aws/credentials` Datei:

```
root@stlrx2540m1-108:~# cat /root/.aws/credentials
[default]
aws_access_key_id = xxxxxxxxxxxx
aws_secret_access_key = xxxxxxxxxxxxxxxxxxxxxxxxx
root@stlrx2540m1-108:~#
```

13. Vergewissern Sie sich, dass der Bucket erreichbar ist.

```
root@stlrx2540m4-01:~# aws s3 –endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 ls kafkasgdbucket1-2
2021-10-29 21:04:18       1388 1
2021-10-29 21:04:20       1388 2
2021-10-29 21:04:22       1388 3
root@stlrx2540m4-01:~#
```

14. Konfigurieren der s3-Sink-Eigenschaftendatei für die s3- und Bucket-Konfiguration

```
root@stlrx2540m1-108:~# cat /data/confluent/confluent-
6.2.0/share/confluent-hub-components/confluentinc-kafka-connect-
s3/etc/quickstart-s3.properties | grep -v ^#
name=s3-sink
connector.class=io.confluent.connect.s3.S3SinkConnector
tasks.max=1
topics=s3_testtopic
s3.region=us-west-2
s3.bucket.name=kafkasgdbucket1-2
store.url=http://kafkasgd.rtpppe.netapp.com:10444/
s3.part.size=5242880
flush.size=3
storage.class=io.confluent.connect.s3.storage.S3Storage
format.class=io.confluent.connect.s3.format.avro.AvroFormat
partitioner.class=io.confluent.connect.storage.partitioner.DefaultPartit
ioner
schema.compatibility=NONE
root@stlrx2540m1-108:~#
```

15. Importieren Sie einige Datensätze in den s3-Bucket.

```
kafka-avro-console-producer --broker-list localhost:9092 --topic
s3_topic \
--property
value.schema='{"type":"record","name":"myrecord","fields":[{"name":"f1",
"type":"string"}]}'
{"f1": "value1"}
{"f1": "value2"}
{"f1": "value3"}
{"f1": "value4"}
{"f1": "value5"}
{"f1": "value6"}
{"f1": "value7"}
{"f1": "value8"}
{"f1": "value9"}
```

16. Den s3-Spülkörper-Anschluss einlegen.

```
root@stlrx2540m1-108:~# confluent local services connect connector load
s3-sink  --config /data/confluent/confluent-6.2.0/share/confluent-hub-
components/confluentinc-kafka-connect-s3/etc/quickstart-s3.properties
The local commands are intended for a single-node development
environment only,
NOT for production usage.
https://docs.confluent.io/current/cli/index.html
{
  "name": "s3-sink",
  "config": {
    "connector.class": "io.confluent.connect.s3.S3SinkConnector",
    "flush.size": "3",
    "format.class": "io.confluent.connect.s3.format.avro.AvroFormat",
    "partitioner.class":
"io.confluent.connect.storage.partitioner.DefaultPartitioner",
    "s3.bucket.name": "kafkasgdbucket1-2",
    "s3.part.size": "5242880",
    "s3.region": "us-west-2",
    "schema.compatibility": "NONE",
    "storage.class": "io.confluent.connect.s3.storage.S3Storage",
    "store.url": "http://kafkasgd.rtpppe.netapp.com:10444/",
    "tasks.max": "1",
    "topics": "s3_testtopic",
    "name": "s3-sink"
  },
  "tasks": [],
  "type": "sink"
}
root@stlrx2540m1-108:~#
```

17. Überprüfen Sie den s3-Sink-Status.

```
root@stlrx2540m1-108:~# confluent local services connect connector
status s3-sink
The local commands are intended for a single-node development
environment only,
NOT for production usage.
https://docs.confluent.io/current/cli/index.html
{
  "name": "s3-sink",
  "connector": {
    "state": "RUNNING",
    "worker_id": "10.63.150.185:8083"
  },
  "tasks": [
    {
      "id": 0,
      "state": "RUNNING",
      "worker_id": "10.63.150.185:8083"
    }
  ],
  "type": "sink"
}
root@stlrx2540m1-108:~#
```

18. Prüfen Sie das Protokoll, um sicherzustellen, dass s3-Sink bereit ist, Themen zu akzeptieren.

```
root@stlrx2540m1-108:~# confluent local services connect log
```

19. Informieren Sie sich in Kafka über die Themen.

```
kafka-topics --list --bootstrap-server localhost:9092
…
connect-configs
connect-offsets
connect-statuses
default_ksql_processing_log
s3_testtopic
s3_topic
s3_topic_new
root@stlrx2540m1-108:~#
```

20. Überprüfen Sie die Objekte im s3-Bucket.

```
root@stlrx2540m1-108:~# aws s3 --endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 ls --recursive kafkasgdbucket1-
2/topics/
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000003.avro
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000006.avro
2021-10-29 21:24:08        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000009.avro
2021-10-29 21:24:08        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000012.avro
2021-10-29 21:24:09        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000015.avro
root@stlrx2540m1-108:~#
```

21. Um den Inhalt zu überprüfen, kopieren Sie jede Datei von S3 in Ihr lokales Dateisystem, indem Sie den folgenden Befehl ausführen:

```
root@stlrx2540m1-108:~# aws s3 --endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 cp s3://kafkasgdbucket1-
2/topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro
tes.avro
download: s3://kafkasgdbucket1-
2/topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro to
./tes.avro
root@stlrx2540m1-108:~#
```

22. Um die Datensätze zu drucken, verwenden Sie avro-tools-1.11.0.1.jar (verfügbar im "Apache Archives").

```
root@stlrx2540m1-108:~# java -jar /usr/src/avro-tools-1.11.0.1.jar
tojson tes.avro
21/10/30 00:20:24 WARN util.NativeCodeLoader: Unable to load native-
hadoop library for your platform... using builtin-java classes where
applicable
{"f1":"value1"}
{"f1":"value2"}
{"f1":"value3"}
root@stlrx2540m1-108:~#
```

"Als Nächstes: Conflient Self-Rebalancing Cluster."

Confluent Self balancing Clusters

"Früher: Kafka s3 Stecker."

Wenn Sie bereits einen Kafka Cluster gemanagt haben, kennen Sie die Herausforderungen, die sich aus der manuellen Neuzuteilung von Partitionen an verschiedene Broker ergeben, um sicherzustellen, dass der Workload über das Cluster hinweg ausgeglichen wird. Für Unternehmen mit großen Implementierungen im Kafka-Bereich kann das Umstellen großer Datenmengen eine enorme Herausforderung, ein mühsames und riskantes sein, vor allem, wenn geschäftskritische Applikationen auf dem Cluster basieren. Selbst für die kleinsten Anwendungsfälle von Kafka ist dieser Vorgang allerdings zeitaufwändig und fehleranfällig.

In unserem Labor haben wir die Funktion Confluent Self-Balancing Cluster getestet, die die Ausbalancierung auf Basis von Cluster-Topologieänderungen oder unausgeglichenen Lasten automatisiert. Der Test zur Neuverteilung bei einem Knoten hilft dabei, die Zeit zu messen, die zum Hinzufügen eines neuen Brokers erforderlich ist, wenn ein Node ausfällt oder der Skalierungsknoten eine Ausbalancierung der Daten zwischen Brokern erfordert. In klassischen Kafka-Konfigurationen wächst die Menge der auszubalancieren, während der Cluster wächst. Bei Tiered Storage ist die Ausbalancierung jedoch auf kleine Datenmengen beschränkt. Basierend auf unserer Validierung dauert die Lastverteilung im Tiered Storage in einer klassischen Kafka-Architektur nur Sekunden oder Minuten und wächst linear mit dem Cluster.

In Self-Balancing-Clustern sind Partitions-Rebalons vollständig automatisiert. Dies optimiert den Durchsatz von Kafka, beschleunigt die Skalierung von Vermittlern und entlastet den Betrieb eines großen Clusters. Bei Clustern mit Selbstausgleich überwachen die Verzerrung der Daten zwischen den Brokern und weist Partitionen kontinuierlich neu zu, um die Cluster-Performance zu optimieren. Bei der Skalierung der Plattform nach oben oder unten erkennen Self-Balancing-Cluster automatisch das Vorhandensein neuer Broker oder das Entfernen alter Broker und lösen eine nachfolgende Neuzuweisung von Partitionen aus. So können Sie problemlos Broker hinzufügen und ausmustern und Ihre Kafka Cluster damit im Wesentlichen flexibler gestalten. Diese Vorteile entstehen ohne manuelles Eingreifen, komplexe Mathematik oder das Risiko menschlicher Fehler, die für gewöhnlich eine Partition neu zugewiesen werden. Auf diese Weise werden Datenabstände in wesentlich kürzerer Zeit abgeschlossen und Sie können sich auf wichtigere Event-Streaming-Projekte konzentrieren, anstatt Ihre Cluster ständig zu überwachen.

"Als Nächstes: Best Practice-Richtlinien."

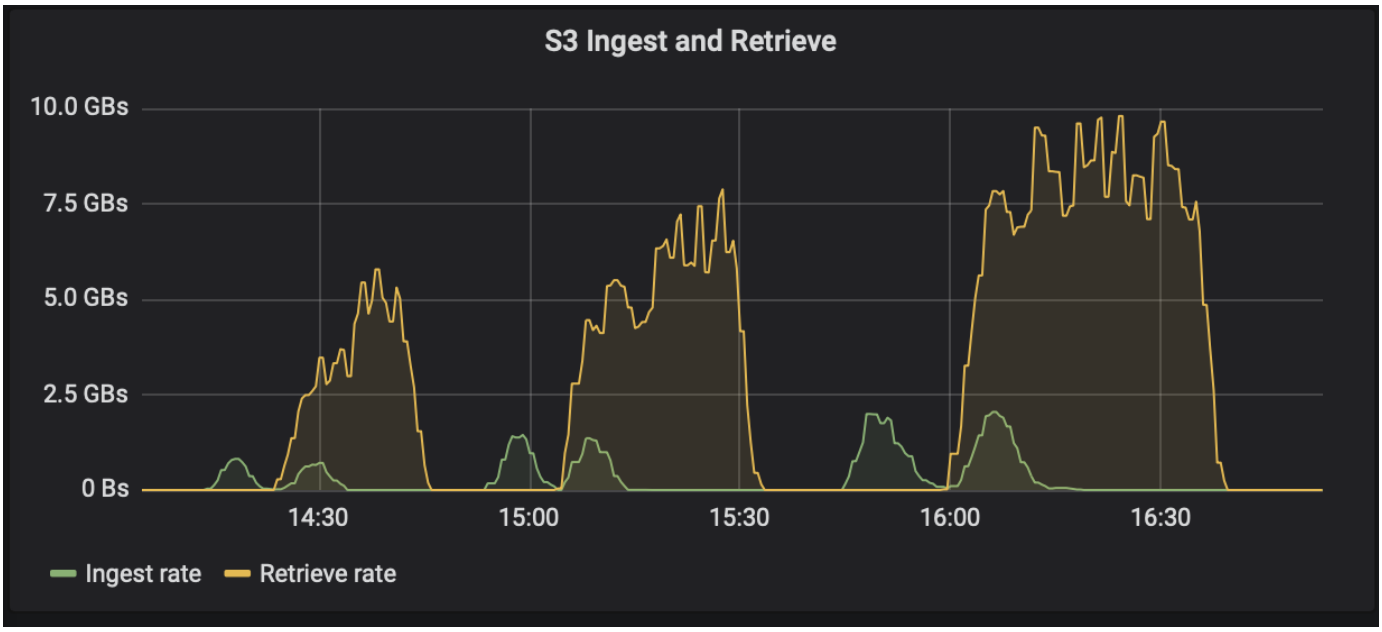
Best Practices-Richtlinien

"Früher: Confluent Self-Rebalancing Cluster."

- Basierend auf unserer Validierung ist S3 Objekt-Storage ideal für Conflient, um Daten zu speichern.
- Wir können SAN mit hohem Durchsatz (speziell FC) verwenden, um den Broker häufig verwendete Daten oder lokale Festplatten zu erhalten, da wir in der Conflient Tiered Storage-Konfiguration, Die Größe der im Datenvermittler-Verzeichnis enthaltenen Daten richtet sich nach der Segmentgröße und der Aufbewahrungszeit, wenn die Daten in den Objekt-Storage verschoben werden.
- Objektspeicher bieten eine bessere Performance, wenn Segment.Bytes höher ist. Wir haben 512 MB getestet.
- In Kafka wird die Länge des Schlüssels oder Wertes (in Bytes) für jeden Datensatz, der zum Thema produziert wird, durch das gesteuert `length.key.value` Parameter. Bei StorageGRID hat sich die Performance bei der Aufnahme und Wiederherstellung von S3-Objekten auf höhere Werte erhöht. 512 Byte sorgte beispielsweise für einen Abruf von 5,8 GB/s, 1024 Bytes sorgten für einen s3-Abruf von 7,5 GB/s und 2048 Bytes für fast 10 GB/s.

In der folgenden Abbildung sind die S3-Objektaufnahme und -Abruf basierend auf dargestellt

`length.key.value`.



- **Kafka Tuning.** um die Performance von Tiered Storage zu verbessern, können Sie TierFetcherNumThreads und TierArchivNumThreads erhöhen. Als allgemeine Richtlinie möchten Sie TierFetcherNumThreads erhöhen, um mit der Anzahl der physischen CPU-Kerne zu übereinstimmen und TierArchivNumThreads auf die Hälfte der CPU-Kerne zu erhöhen. Wenn Sie zum Beispiel in den Servereigenschaften eine Maschine mit acht physischen Kernen haben, setzen Sie confluent.Tier.fetcher.num.threads = 8 und confluent.Tier.archiver.num.threads = 4.
- **Zeitintervall für Thema löscht.** Wenn ein Thema gelöscht wird, beginnt das Löschen der Log-Segment-Dateien im Objektspeicher nicht sofort. Stattdessen gibt es ein Zeitintervall mit einem Standardwert von 3 Stunden, bevor das Löschen dieser Dateien erfolgt. Sie können die Konfiguration confluent.tier.topic.delete.check.interval.ms ändern, um den Wert dieses Intervalls zu ändern. Wenn Sie ein Thema oder Cluster löschen, können Sie die Objekte auch manuell im jeweiligen Bucket löschen.
- **ACLs auf internen Tiered Storage-Themen.** eine Best Practice für On-Premises-Implementierungen ist die Aktivierung einer ACL-Autorisierung für die internen Themen für Tiered Storage. Legen Sie ACL-Regeln fest, um den Zugriff auf diese Daten nur dem Broker-Benutzer zu beschränken. Dies sichert interne Themen und verhindert nicht autorisierten Zugriff auf Tiered Storage-Daten und Metadaten.

```
kafka-acls --bootstrap-server localhost:9092 --command-config adminclient-
configs.conf \
--add --allow-principal User:<kafka> --operation All --topic "_confluent-
tier-state"
```

Ersetzen Sie den Benutzer `<kafka>` Mit dem eigentlichen Brokerchef in Ihrer Implementierung

Beispiel: Der Befehl `confluent-tier-state` Legt ACLs zum internen Thema für Tiered Storage fest. Derzeit gibt es nur ein einziges internes Thema im Zusammenhang mit Tiered Storage. Das Beispiel erstellt eine ACL, die die Hauptberechtigung von Kafka für alle Vorgänge im internen Thema erhält.

"Als Nächstes: Dimensionierung."

Dimensionierung

"Früher: Best Practice-Richtlinien."

Kafka Sizing kann mit vier Konfigurationsmodi durchgeführt werden: Einfache, granulare, Reverse und Partitionen.

Einfach

Der einfache Modus eignet sich für die ersten Apache Kafka-Anwender oder für die frühen Anwendungsfälle. Für diesen Modus stellen Sie Anforderungen wie Durchsatz MB/s, Lese-Fanout, Aufbewahrung und den Prozentsatz der Ressourcenauslastung bereit (standardmäßig 60 %). Auch die Umgebungen wechseln wie beispielsweise On-Premises (Bare-Metal, VMware, Kubernetes oder OpenStack) oder der Cloud. Basierend auf diesen Informationen bietet die Größenbemessung eines Kafka-Clusters die Anzahl der Server, die für den Broker, den Zookeeper, Apache Kafka Connect Workers, die Schemaregistrierung, einen REST Proxy, ksqlDB und das Confluent Control Center erforderlich sind.

Bei Tiered Storage empfiehlt es sich, den granularen Konfigurationsmodus zu verwenden, um einen Kafka Cluster zu bedimensionieren. Der granulare Modus eignet sich für erfahrene Anwender von Apache Kafka oder für klar definierte Anwendungsfälle. In diesem Abschnitt wird die Größe für Hersteller, Stream-Prozessoren und Verbraucher beschrieben.

Hersteller

Um die Hersteller für Apache Kafka zu beschreiben (z. B. ein nativer Client, REST Proxy oder Kafka Connector), geben Sie folgende Informationen an:

- **Name** Funke.
- **Producer-Typ.** Anwendung oder Dienst, Proxy (REST, MQTT, andere) und bestehende Datenbank (RDBMS, NOSQL, andere). Sie können auch wählen, "Ich weiß nicht."
- **Durchschnittlicher Durchsatz.** bei Ereignissen pro Sekunde (z. B. 1,000,000).
- * Spitzendurchsatz.* bei Ereignissen pro Sekunde (4,000,000 zum Beispiel).
- **Durchschnittliche Nachrichtengröße.** in Byte, unkomprimiert (max. 1 MB; z. B. 1000).
- **Nachrichtenformat.** Optionen umfassen Avro, JSON, Protokollpuffer, Binärdateien, Text, „Ich weiß es nicht“ und andere.
- **Replikationsfaktor.** Optionen sind 1, 2, 3 (Confluent-Empfehlung), 4, 5, Oder 6.
- **Aufbewahrungszeit.** eines Tages (zum Beispiel). Wie lange sollen Ihre Daten in Apache Kafka gespeichert werden? Geben Sie -1 mit einer beliebigen Einheit für eine unendliche Zeit ein. Der Rechner nimmt eine Aufbewahrungszeit von 10 Jahren für unbegrenzte Aufbewahrung an.
- Aktivieren Sie das Kontrollkästchen „Tiered Storage ermöglichen, um die Broker-Anzahl zu verringern und Infinite Storage zu ermöglichen?“
- Wenn Tiered Storage aktiviert ist, kontrollieren die Aufbewahrungsfelder den Hot Set von Daten, die lokal auf dem Broker gespeichert werden. Die Felder zur Aufbewahrung von Archivierung steuern, wie lange Daten im Objekt-Storage für die Archivierung gespeichert werden.
- **Archivierung Speicherbindung.** ein Jahr (zum Beispiel). Wie lange möchten Sie, dass Ihre Daten im Archiv-Storage gespeichert werden? Geben Sie -1 mit einer beliebigen Einheit für eine unbegrenzte Dauer ein. Der Rechner geht von einer Aufbewahrung von 10 Jahren für unbegrenzte Aufbewahrung aus.
- **Wachstumsmultiplikator.** 1 (zum Beispiel). Wenn der Wert dieses Parameters auf dem aktuellen Durchsatz basiert, setzen Sie ihn auf 1. Wenn Sie die Größe basierend auf einem zusätzlichen Wachstum

festlegen möchten, setzen Sie diesen Parameter auf einen Wachstumsmultiplikator.

- **Anzahl der Erzeugerinstanzen.** 10 (zum Beispiel). Wie viele Erzeugerinstanzen werden ausgeführt? Diese Eingabe ist erforderlich, um die CPU-Last in die Berechnung der Größenbemessung einzubeziehen. Ein Leerwert bedeutet, dass die CPU-Last nicht in die Berechnung einbezogen wird.

Anhand dieses Beispieleingangs hat die Größenbemessung folgende Auswirkungen auf die Hersteller:

- Durchschnittlicher Durchsatz in unkomprimierten Bytes: 1 Gbit/s Spitzendurchsatz in unkomprimierten Bytes: 4 GBit/s. Durchschnittlicher Durchsatz in komprimierten Bytes: 400 MBit/s. Spitzendurchsatz in komprimierten Bytes: 1,6 GBit/s. Dies basiert auf einer standardmäßigen Komprimierungsrate von 60 % (Sie können diesen Wert ändern).
  - Insgesamt erforderlicher Storage für Brokersatz: 31,104 TB einschließlich Replizierung, komprimiert Insgesamt erforderlicher Off-Broker-Archiv-Storage: 378,432 TB, komprimiert Nutzung "https://fusion.netapp.com" Für die StorageGRID-Dimensionierung.

Stream-Prozessoren müssen ihre Applikationen oder Services beschreiben, die Daten aus Apache Kafka verbrauchen und dann in Apache Kafka zurückproduzieren. In den meisten Fällen sind diese in ksqlDB oder Kafka-Bächen gebaut.

- **Name** Funkenstreamer.
- **Verarbeitungszeit.** wie lange dauert dieser Prozessor, um eine einzelne Nachricht zu verarbeiten?
  - 1 ms (einfache, statusfreie Transformation) [Beispiel], 10 ms (statusfreier in-Memory-Betrieb).
  - 100 ms (statusbehafteter Netzwerk- oder Festplattenbetrieb), 1000 ms (RUHEZUSTAND eines Drittanbieters).
  - Ich habe diesen Parameter bewertet und weiß genau, wie lange es dauert.
- **Aufbewahrung der Ausgabe.** 1 Tag (Beispiel). Ein Stream-Prozessor produziert seine Ausgabe zurück an Apache Kafka. Wie lange sollen diese Ausgabedaten in Apache Kafka gespeichert werden? Geben Sie -1 mit einer beliebigen Einheit für eine unbegrenzte Dauer ein.
- Aktivieren Sie das Kontrollkästchen „Tiered Storage ermöglichen, um die Broker-Anzahl zu verringern und Infinite Storage zu ermöglichen?“
- **Archivierung Speicherbindung.** 1 Jahr (zum Beispiel). Wie lange möchten Sie, dass Ihre Daten im Archiv-Storage gespeichert werden? Geben Sie -1 mit einer beliebigen Einheit für eine unbegrenzte Dauer ein. Der Rechner geht von einer Aufbewahrung von 10 Jahren für unbegrenzte Aufbewahrung aus.
- **Output Passthrough Prozentsatz.** 100 (zum Beispiel). Ein Stream-Prozessor produziert seine Ausgabe zurück an Apache Kafka. Welcher Prozentsatz des eingehenden Durchsatzes wird in Apache Kafka zurückeingegeben? Beispiel: Wenn der Eingangsdurchsatz 20 Mbit/s beträgt und dieser Wert 10 ist, beträgt der Ausgangsdurchsatz 2 Mbit/s.
- Von welchen Anwendungen wird das gelesen? Wählen Sie „Spark“, den Namen, der in der herstellerspezifischen Dimensionierung verwendet wird. Basierend auf dem obigen Input, können Sie die folgenden Auswirkungen der Dimensionierung auf Stream-Prozessor-Instanzen und Themen-Partition Schätzungen erwarten:
- Diese Stream-Prozessor-Anwendung erfordert die folgende Anzahl von Instanzen. Auch für die eingehenden Themen sind diese vielen Partitionen wahrscheinlich erforderlich. Kontaktieren Sie Confluent, um diesen Parameter zu bestätigen.
  - 1,000 für durchschnittlichen Durchsatz ohne Wachstumsmultiplikator
  - 4,000 für Spitzendurchsatz ohne Wachstumsmultiplikator
  - 1,000 für durchschnittlichen Durchsatz mit einem Wachstumsmultiplikator

    - 4,000 für Spitzendurchsatz mit einem Wachstumsmultiplikator

Verbraucher

Beschreiben Sie Ihre Applikationen oder Services, die Daten aus Apache Kafka verbrauchen und keine Daten zurück in Apache Kafka liefern – beispielsweise einen nativen Client oder Kafka Connector.

- **Name** Kunde von Spark.
- **Bearbeitungszeit.** wie lange dauert der Verbraucher, um eine einzelne Nachricht zu verarbeiten?
    - 1 ms (z. B. eine einfache und statusfreie Aufgabe wie Protokollierung)
    - 10 ms (schnelles Schreiben auf einen Datenspeicher)
    - 100 ms (langsame Schreibvorgänge auf einen Datenspeicher)
    - 1000 ms (ANRUF eines Drittanbieters)
    - Ein anderer Benchmark-Prozess bekannter Dauer.
- **Verbrauchertyp** Anwendung, Proxy oder auf einen vorhandenen Datastore absinken (RDBMS, NoSQL, andere).
- Von welchen Anwendungen wird das gelesen? Schließen Sie diesen Parameter an Hersteller und Stream-Größen an, die zuvor ermittelt wurden.

Anhand der obigen Angaben müssen Sie die Größe für Verbraucherinstanzen und Schätzungen der Themenpartition ermitteln. Eine Verbraucheranwendung benötigt die folgende Anzahl von Instanzen.

- 2,000 für durchschnittlichen Durchsatz, kein Wachstumsmultiplikator
- 8,000 für Spitzendurchsatz, kein Wachstumsmultiplikator
- 2,000 für den durchschnittlichen Durchsatz, einschließlich Wachstums Multiplikator
- 8,000 für Spitzendurchsatz, einschließlich Wachstumsmultiplikator

Auch für die eingehenden Themen ist diese Anzahl von Partitionen wahrscheinlich erforderlich. Kontaktieren Sie Confluent, um die Bestätigung zu bestätigen.

Zusätzlich zu den Anforderungen für Hersteller, Stream-Prozessoren und Verbraucher müssen Sie die folgenden zusätzlichen Anforderungen erfüllen:

- **Wiederherstellungszeit.** zum Beispiel 4 Stunden. Wenn ein Apache Kafka Broker-Host ausfällt, gehen seine Daten verloren und ein neuer Host wird bereitgestellt, um den ausgefallenen Host zu ersetzen. Wie schnell muss dieser neue Host neu erstellt werden? Lassen Sie diesen Parameter leer, wenn der Wert unbekannt ist.
- **Ressourcenausnutzungsziel (Prozentsatz).** zum Beispiel 60. Wie ausgelastet sollen Ihre Hosts im Durchschnitt des Durchsatzes sein? Conflient empfiehlt eine Auslastung von 60 %, es sei denn, Sie verwenden Conflient Self-Balancing Cluster, in einem solchen Fall kann die Auslastung höher sein.

Beschreiben Sie Ihre Umgebung

- * Welche Umgebung wird Ihr Cluster ausführen?* Amazon Web Services, Microsoft Azure, Google Cloud Platform, Bare-Metal On Premises, VMware On Premises, OpenStack vor Ort oder Kubernates vor Ort?
- **Hostdetails.** Anzahl der Kerne: 48 (zum Beispiel), Netzwerkkartentyp (10 GbE, 40 GbE, 16 GbE, 1 GbE oder ein anderer Typ).
- **Speichervolumen.** Host: 12 (zum Beispiel). Wie viele Festplatten oder SSDs werden pro Host unterstützt?

Confluent empfiehlt 12 Festplatten pro Host.

- **Speicherkapazität/Volumen (in GB).** 1000 (zum Beispiel). Wie viel Storage kann ein einzelnes Volume in Gigabyte speichern? Fließend empfiehlt 1-TB-Festplatten.
- **Speicherkonfiguration.** wie werden Storage Volumes konfiguriert? Confluent empfiehlt RAID10, alle Confluent Funktionen zu nutzen. JBOD, SAN, RAID 1, RAID 0, RAID 5, Weitere Typen werden ebenfalls unterstützt.
- **Single Volume Throughput (Mbps).** 125 (zum Beispiel). Wie schnell kann ein einzelnes Storage Volume in Megabyte pro Sekunde lesen oder schreiben? Confluent empfiehlt Standard-Festplatten, die in der Regel einen Durchsatz von 125 MB/s haben.
- **Speicherkapazität (GB).** 64 (zum Beispiel).

Wählen Sie Größe mein Cluster aus, nachdem Sie Ihre Umgebungsvariablen festgelegt haben. Anhand der oben genannten Beispielparameter haben wir für Confluent Kafka folgende Bemessungsparameter ermittelt:

- **Apache Kafka.** Broker zählen: 22. Der Cluster ist Storage-gebunden. Erwägen Sie die Aktivierung von Tiered Storage zur Reduzierung Ihrer Host-Anzahl und für skalierbaren Storage.
- **Apache ZooKeeper.** Anzahl: 5; Apache Kafka Connect Workers: Anzahl: 2; Schema Registry: Anzahl: 2; REST Proxy: Anzahl: 2; ksqlDB: Anzahl: 2; Confluent Control Center: Anzahl: 1.

Für Plattform-Teams ohne Anwendungsfälle sollte der Reverse-Modus genutzt werden. Verwenden Sie den Partitions-Modus, um zu berechnen, wie viele Partitionen ein einzelnes Thema benötigt. Siehe https://eventsizer.io Für die Dimensionierung basierend auf dem Reverse-Modus und den Partitions-Modus.

"Weiter: Fazit."

Schlussfolgerung

"Zurück: Dimensionierung."

Dieses Dokument enthält Best Practices für die Verwendung von Confluent Tiered Storage mit NetApp Storage, einschließlich Verifizierungstests, Tiered Storage-Performance-Ergebnissen, Tuning, Conflient S3-Konnektoren und der Self-Balancing-Funktion. Der NetApp StorageGRID Objekt-Storage zieht ILM-Richtlinien in Betracht, bietet fließende Performance mit mehreren Performance-Tests zur Verifizierung und S3 APIs nach Branchenstandard. Dies ist eine optimale Wahl für Conflient Tiered Storage.

Wo Sie weitere Informationen finden

Sehen Sie sich die folgenden Dokumente und/oder Websites an, um mehr über die in diesem Dokument beschriebenen Informationen zu erfahren:

- Was ist Apache Kafka

  "https://www.confluent.io/what-is-apache-kafka/"

- NetApp Produktdokumentation

  "https://www.netapp.com/support-and-training/documentation/"

- S3-Sink Parameterdetails

  "https://docs.confluent.io/kafka-connect-s3-sink/current/configuration_options.html#s3-configuration-options"

- Apache Kafka

  "https://en.wikipedia.org/wiki/Apache_Kafka"

- Infinite Storage in Conflient Plattform

  "https://www.confluent.io/blog/infinite-kafka-storage-in-confluent-platform/"

- Confluent Tiered Storage – Best Practices und Sizing

  "https://docs.confluent.io/platform/current/kafka/tiered-storage.html#best-practices-and-recommendations"

- Amazon S3 Spülenanschluss für Confluent Platform

  "https://docs.confluent.io/kafka-connect-s3-sink/current/overview.html"

- Kafka – Größen

  "https://eventsizer.io"

- StorageGRID Dimensionierung

  "https://fusion.netapp.com/"

- Anwendungsfälle für Kafka

  "https://kafka.apache.org/uses"

- Kafka-Cluster mit Selbstausgleich in Plattform 6.0

  "https://www.confluent.io/blog/self-balancing-kafka-clusters-in-confluent-platform-6-0/"

  "https://www.confluent.io/blog/confluent-platform-6-0-delivers-the-most-powerful-event-streaming-platform-to-date/"

Versionsverlauf

Version	Datum	Versionsverlauf des Dokuments
Version 1.0	Dezember 2021	Erste Version.

Copyright-Informationen

Copyright © 2022 NetApp. Alle Rechte vorbehalten. Gedruckt in den USA. Dieses urheberrechtlich geschützte Dokument darf ohne die vorherige schriftliche Genehmigung des Urheberrechtsinhabers in keiner Form und durch keine Mittel – weder grafische noch elektronische oder mechanische, einschließlich Fotokopieren, Aufnehmen oder Speichern in einem elektronischen Abrufsystem – auch nicht in Teilen, vervielfältigt werden.

Software, die von urheberrechtlich geschütztem NetApp Material abgeleitet wird, unterliegt der folgenden Lizenz und dem folgenden Haftungsausschluss:

DIE VORLIEGENDE SOFTWARE WIRD IN DER VORLIEGENDEN FORM VON NETAPP ZUR VERFÜGUNG GESTELLT, D. H. OHNE JEGLICHE EXPLIZITE ODER IMPLIZITE GEWÄHRLEISTUNG, EINSCHLIESSLICH, JEDOCH NICHT BESCHRÄNKT AUF DIE STILLSCHWEIGENDE GEWÄHRLEISTUNG DER MARKTGÄNGIGKEIT UND EIGNUNG FÜR EINEN BESTIMMTEN ZWECK, DIE HIERMIT AUSGESCHLOSSEN WERDEN. NETAPP ÜBERNIMMT KEINERLEI HAFTUNG FÜR DIREKTE, INDIREKTE, ZUFÄLLIGE, BESONDERE, BEISPIELHAFTE SCHÄDEN ODER FOLGESCHÄDEN (EINSCHLIESSLICH, JEDOCH NICHT BESCHRÄNKT AUF DIE BESCHAFFUNG VON ERSATZWAREN ODER -DIENSTLEISTUNGEN, NUTZUNGS-, DATEN- ODER GEWINNVERLUSTE ODER UNTERBRECHUNG DES GESCHÄFTSBETRIEBS), UNABHÄNGIG DAVON, WIE SIE VERURSACHT WURDEN UND AUF WELCHER HAFTUNGSTHEORIE SIE BERUHEN, OB AUS VERTRAGLICH FESTGELEGTER HAFTUNG, VERSCHULDENSUNABHÄNGIGER HAFTUNG ODER DELIKTSHAFTUNG (EINSCHLIESSLICH FAHRLÄSSIGKEIT ODER AUF ANDEREM WEGE), DIE IN IRGENDEINER WEISE AUS DER NUTZUNG DIESER SOFTWARE RESULTIEREN, SELBST WENN AUF DIE MÖGLICHKEIT DERARTIGER SCHÄDEN HINGEWIESEN WURDE.

NetApp behält sich das Recht vor, die hierin beschriebenen Produkte jederzeit und ohne Vorankündigung zu ändern. NetApp übernimmt keine Verantwortung oder Haftung, die sich aus der Verwendung der hier beschriebenen Produkte ergibt, es sei denn, NetApp hat dem ausdrücklich in schriftlicher Form zugestimmt. Die Verwendung oder der Erwerb dieses Produkts stellt keine Lizenzierung im Rahmen eines Patentrechts, Markenrechts oder eines anderen Rechts an geistigem Eigentum von NetApp dar.

Das in diesem Dokument beschriebene Produkt kann durch ein oder mehrere US-amerikanische Patente, ausländische Patente oder anhängige Patentanmeldungen geschützt sein.

ERLÄUTERUNG ZU „RESTRICTED RIGHTS“: Nutzung, Vervielfältigung oder Offenlegung durch die US-Regierung unterliegt den Einschränkungen gemäß Unterabschnitt (b)(3) der Klausel „Rights in Technical Data – Noncommercial Items“ in DFARS 252.227-7013 (Februar 2014) und FAR 52.227-19 (Dezember 2007).

Die hierin enthaltenen Daten beziehen sich auf ein kommerzielles Produkt und/oder einen kommerziellen Service (wie in FAR 2.101 definiert) und sind Eigentum von NetApp, Inc. Alle technischen Daten und die Computersoftware von NetApp, die unter diesem Vertrag bereitgestellt werden, sind gewerblicher Natur und wurden ausschließlich unter Verwendung privater Mittel entwickelt. Die US-Regierung besitzt eine nicht ausschließliche, nicht übertragbare, nicht unterlizenzierbare, weltweite, limitierte unwiderrufliche Lizenz zur Nutzung der Daten nur in Verbindung mit und zur Unterstützung des Vertrags der US-Regierung, unter dem die Daten bereitgestellt wurden. Sofern in den vorliegenden Bedingungen nicht anders angegeben, dürfen die Daten ohne vorherige schriftliche Genehmigung von NetApp, Inc. nicht verwendet, offengelegt, vervielfältigt, geändert, aufgeführt oder angezeigt werden. Die Lizenzrechte der US-Regierung für das US-Verteidigungsministerium sind auf die in DFARS-Klausel 252.227-7015(b) (Februar 2014) genannten Rechte beschränkt.

Markeninformationen

NETAPP, das NETAPP Logo und die unter http://www.netapp.com/TM aufgeführten Marken sind Marken von NetApp, Inc. Andere Firmen und Produktnamen können Marken der jeweiligen Eigentümer sein.